

CS10102302

UML与用例建模

赵君峤 长聘副教授 计算机
科学与技术学院

同济大学

什么是UML？

- UML是**统一建模语言**的缩写。
- UML是一种用于
 - 可视化 (可视化)
 - 规范 (详述)
 - 构建 (构造)
 - 文档化

UML是事实上的标准
用于软件设计与分析的符号体系...

软件密集型系统的技术成果。



布奇



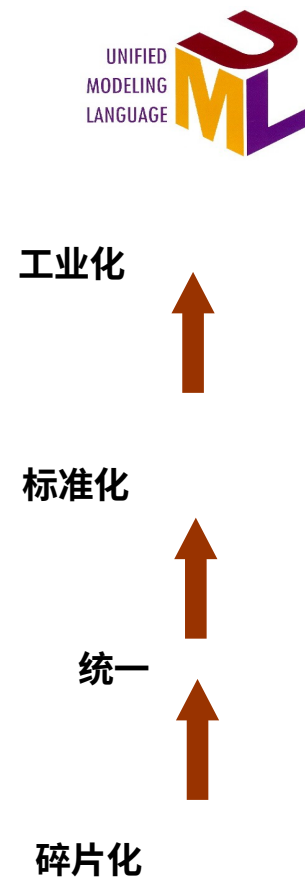
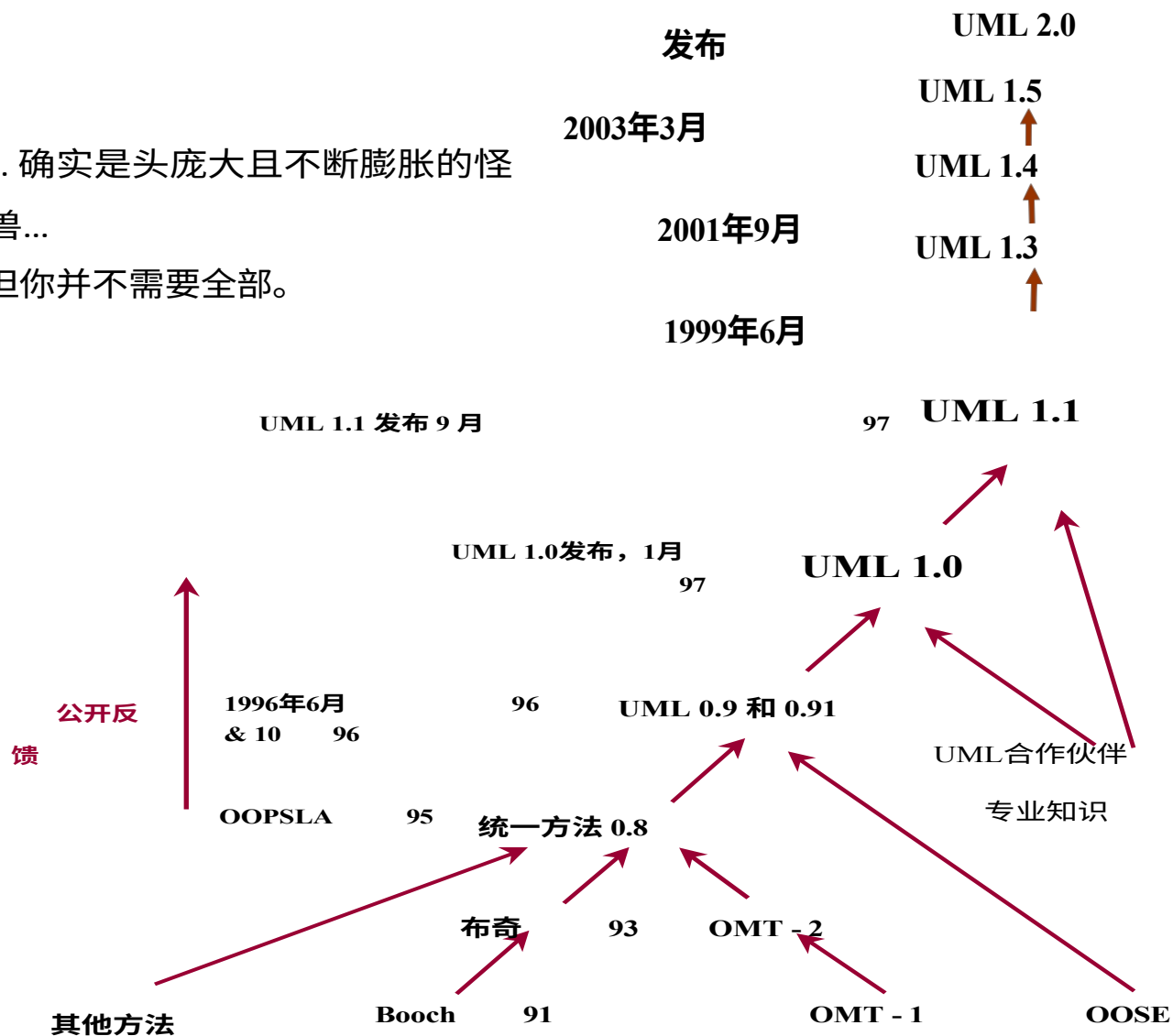
伦博



雅各布森



... 确实是头庞大且不断膨胀的怪兽...
但你并不需要全部。



对UML的贡献



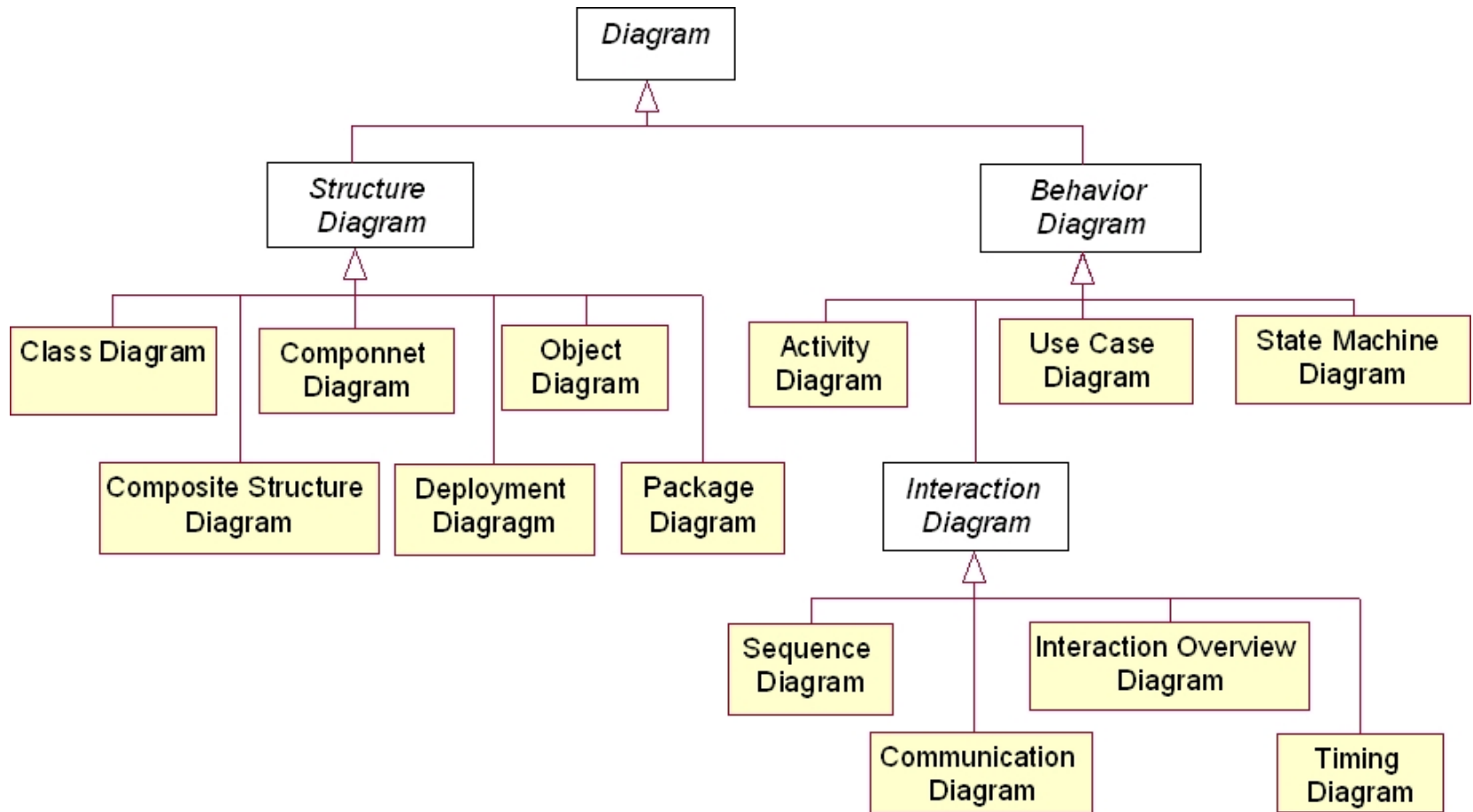
UML构造块

- 基本构造块 (基本构建块)
 - 事物 (things)
 - 结构事物 (structural things)
 - 类、接口、协作、用例、活动类、组件、节点
 - 行为事物 (行为事物)
 - 交互、状态机
 - 分组事物 (grouping things)
 - 包
 - 注释事物 (annotation things)
 - 注释事物 (annotation things)
 - 关系 (relationships)
 - 依赖 (dependency)
 - 关联 (association)
 - 泛化 (generalization)
 - 实现 (realization)
 - 图 (diagrams)
- 规则 (规则)
- 公共机制

... UML图

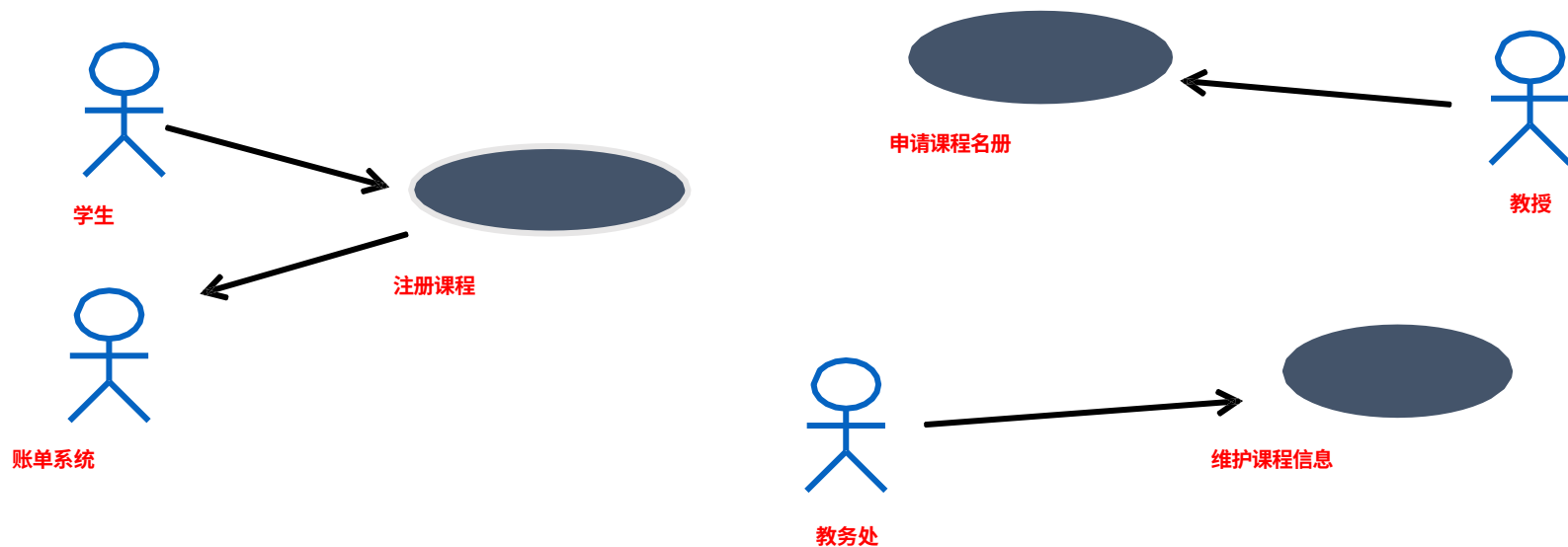
- UML最初被设计为软件建模语言。由于其涵盖需求领域，UML支持世界建模（至少在某种程度上）。
- UML提供多种图示符号体系，用于建模应用程序的静态与动态特性。
- 符号列表包括用例图、类图、交互图——描述事件序列、包图、活动图、状态图、...更多...

UML 2.0 中的图分类



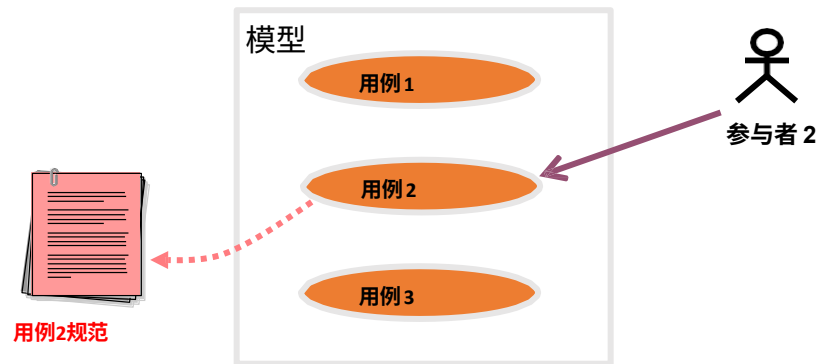
用例图

- 用例图用于可视化系统与外部世界的交互。

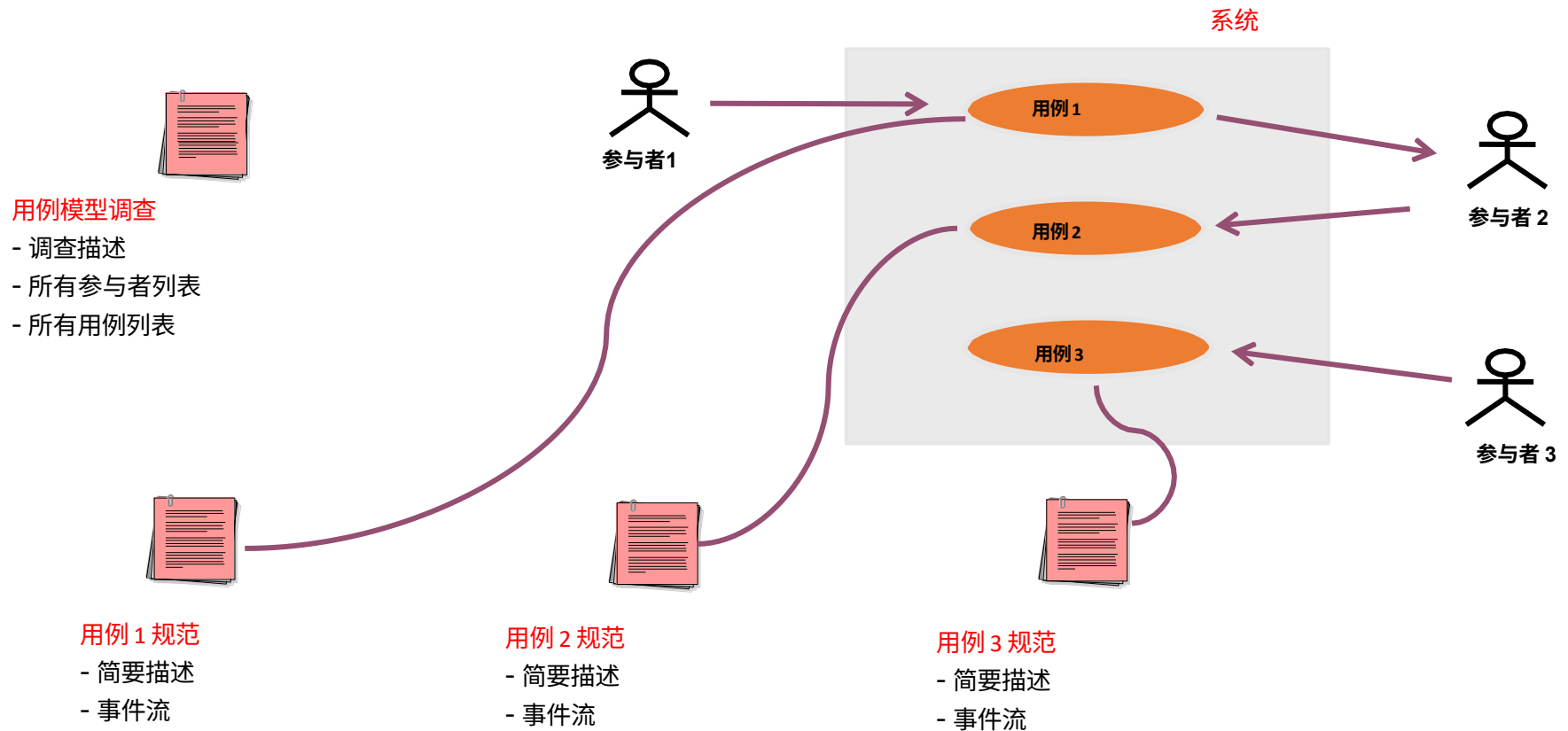


什么是用例建模？

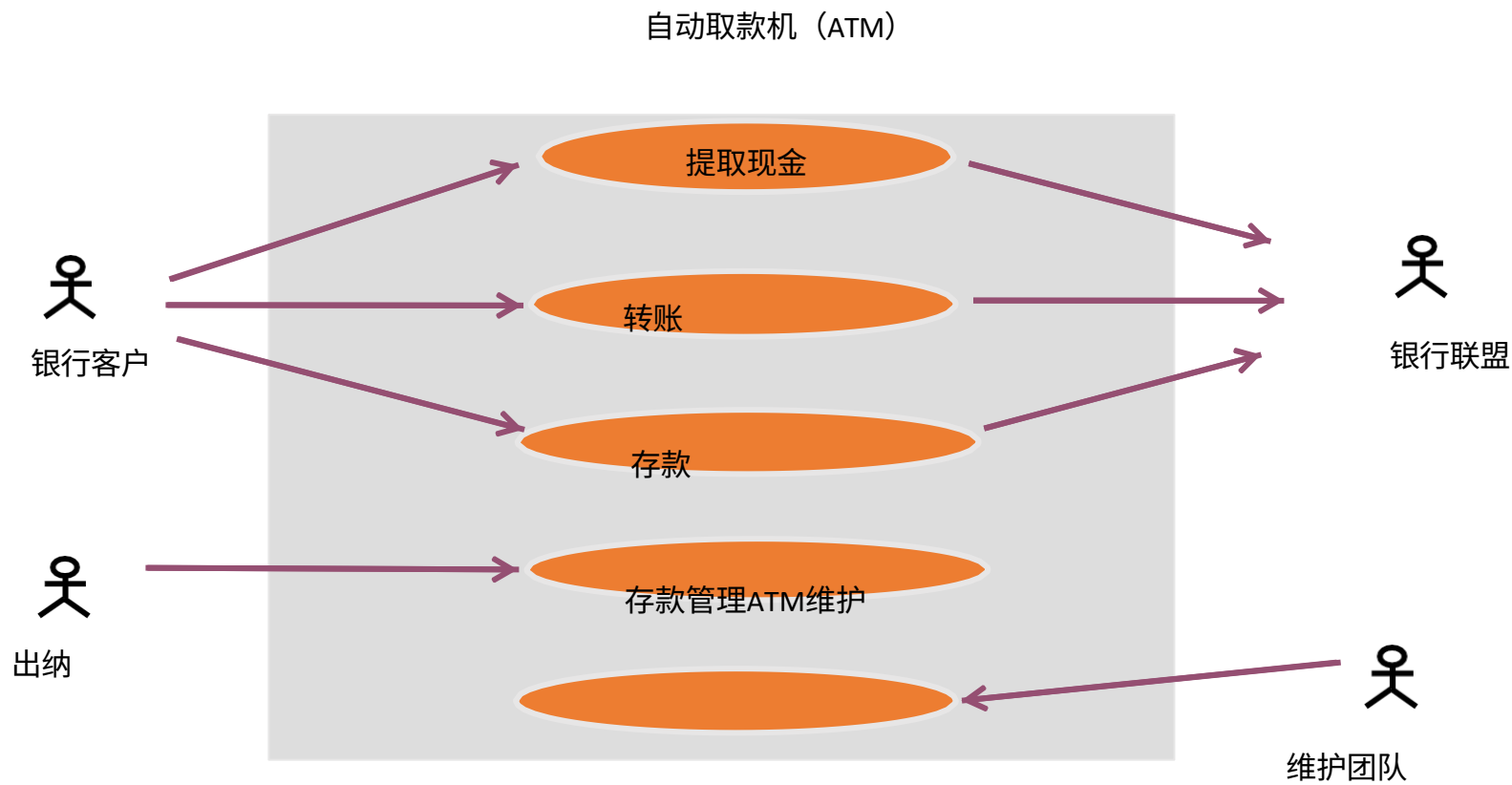
- 将利益相关者的需求与软件需求关联起来。
- 明确定义系统边界。
- 捕捉并传达系统的预期行为。
- 识别与系统交互的对象或主体。
- 验证/确认需求。
- 是一种规划工具。



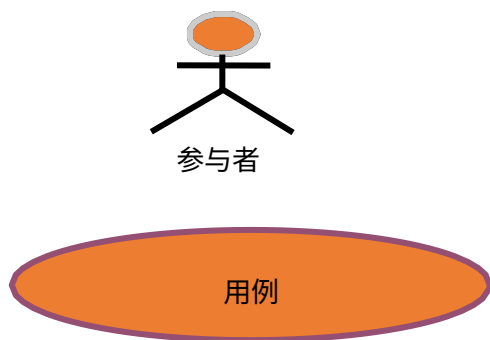
用例模型主要由文本构成



用例图



主要用例建模元素



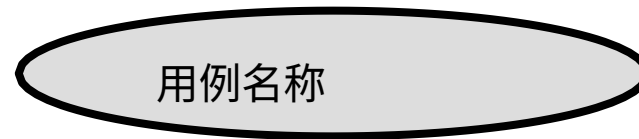
参与者

系统外部的某人/某物，以与系统交互的角色行事

用例

代表系统为其参与者创造的价值

什么是用例?



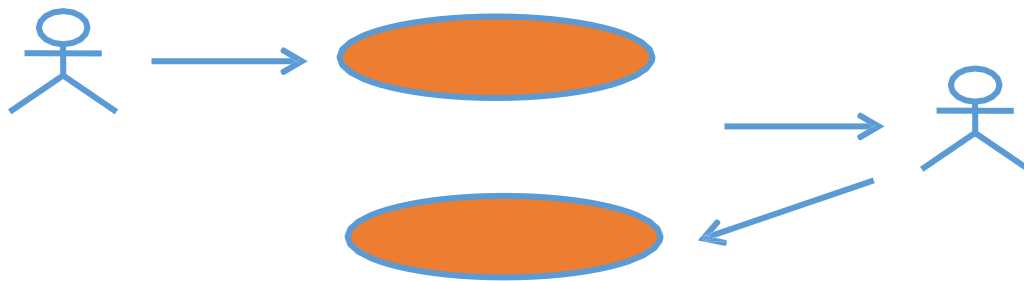
用例

定义系统执行的操作序列

该序列可为参与者产生具有价值的可观察结果。

用例包含软件需求

- 每个用例
 - 描述系统为向参与者交付价值所采取的**行动**。
 - 展示参与者使用的**系统功能**。
 - 建模系统与参与者之间的**对话**。
 - 从特定参与者视角出发，构成完整且有意义的**事件流**。

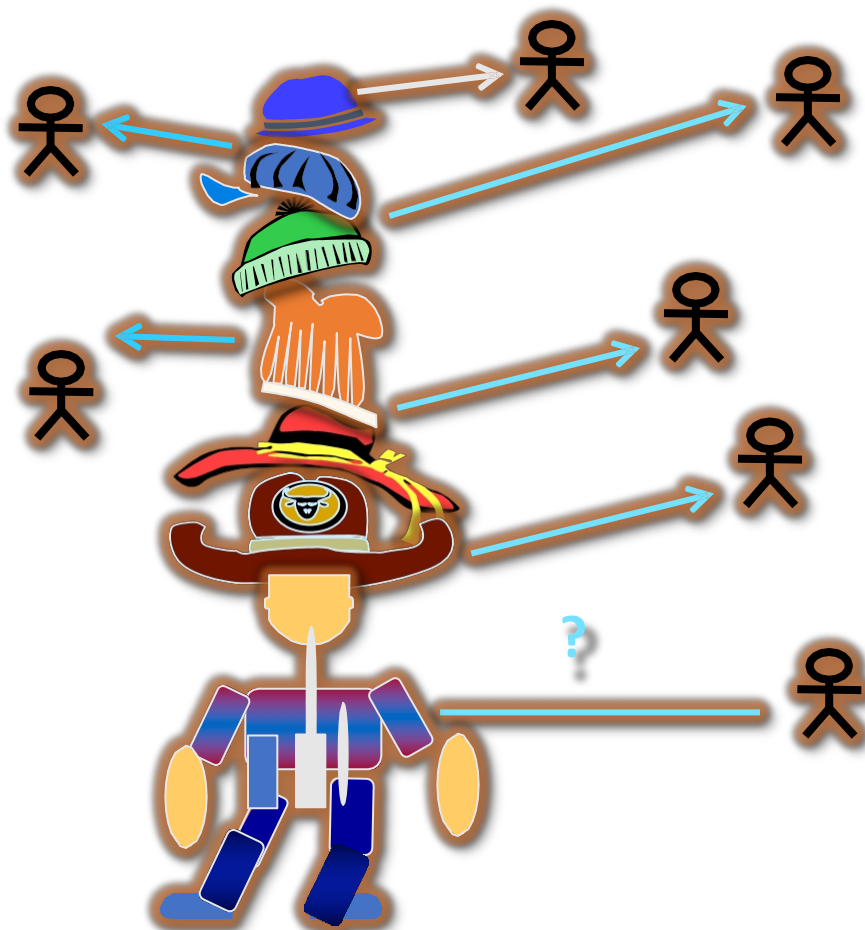


用例的优势

- 为需求提供背景信息。
 - 将系统需求置于逻辑序列中。
 - 阐明系统存在的必要性。
 - 帮助验证是否已捕获所有需求。
- 易于理解。
 - 使用客户和用户理解的术语。
 - 讲述系统使用的具体案例。
 - 验证利益相关者的理解程度。
- 促进与客户达成共识。
- 促进复用：测试、文档和设计。

定义参与者：聚焦角色

- 参与者代表人类、硬件设备或其他系统在系统中扮演的角色。
- 参与者名称应清晰表明其角色。



参与者与角色



查理：受聘为数学教授，同时拥有经济学学士学位。



乔迪：攻读理学学士学位。

查理和乔迪均扮演学生角色。

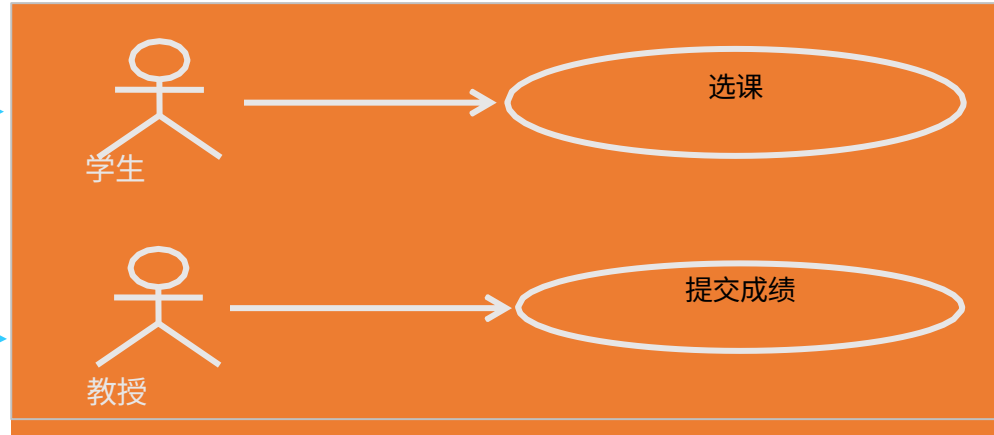


选课

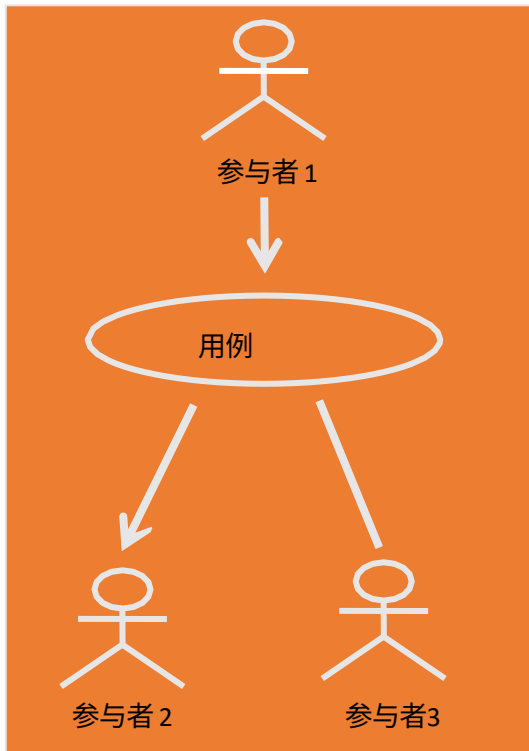
查理还担任教授职务。



提交成绩

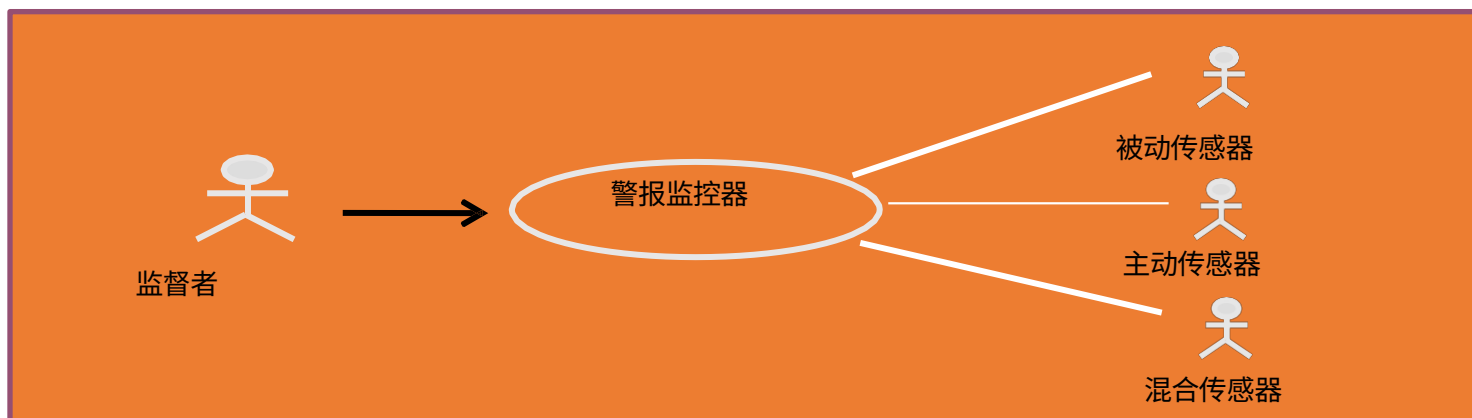
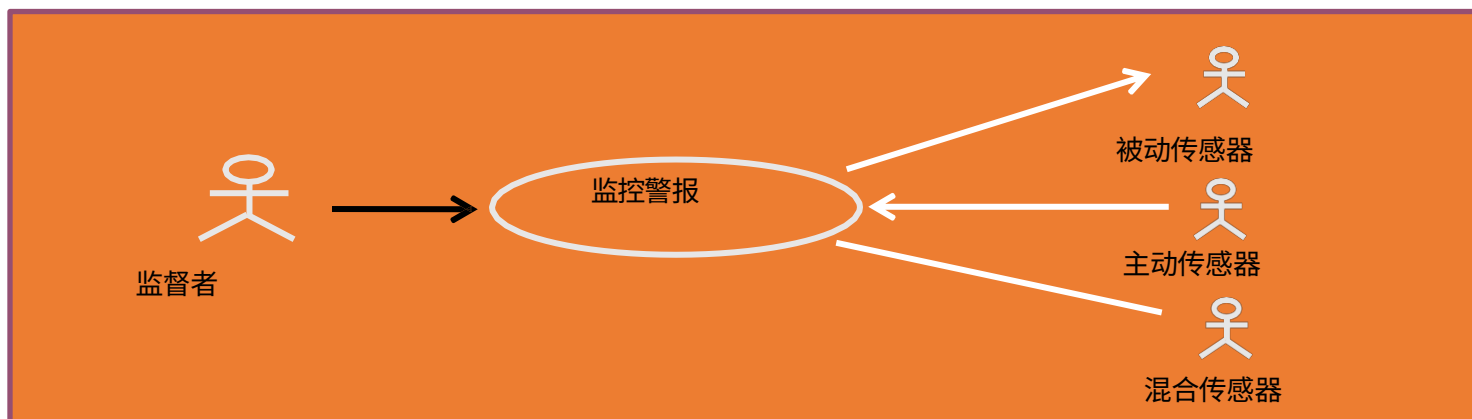


通信-关联

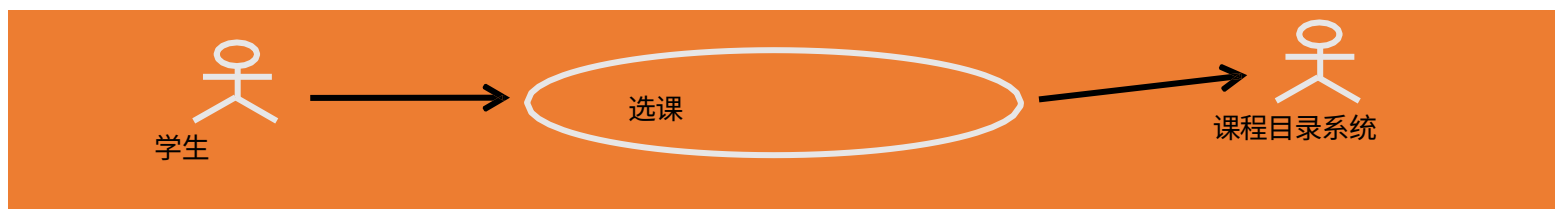


- 演员与用例之间的沟通渠道。
- 使用线条表示通信关联。
 - 箭头指示发起交互的一方。
 - 无箭头表示任一端均可发起交互。

箭头约定



场景即用例实例



情景1

登录系统。批准登录

。在搜索框输入科目。
获取课程列表。显示
课程列表。
选定课程。
确认课程可用性。显示最
终课表。

场景2

登录系统。批准登录

。在搜索中输入科目。
**科目无效。请重新输
入科目。**
获取课程列表。
显示课程列表。选择
课程。
确认可用性。
显示最终课表。

如何命名用例？

- 表明参与者的价值或目标。
- 使用主动语态；以动词开头。
- 想象一份待办事项清单。
- 变体示例
 - 注册课程
 - 选课
 - 确认注册
 - 课程注册
 - 使用注册系统

哪些变体体现了对行为者的价值？哪些没有体现？

你会选择哪个作为用例名称？为什么？

创建用例模型的步骤

◇ 确定参与者和用例。

- ◇ 识别并简要描述参与者。

- ◇ 识别并简要描述用例。

◇ 编写用例。

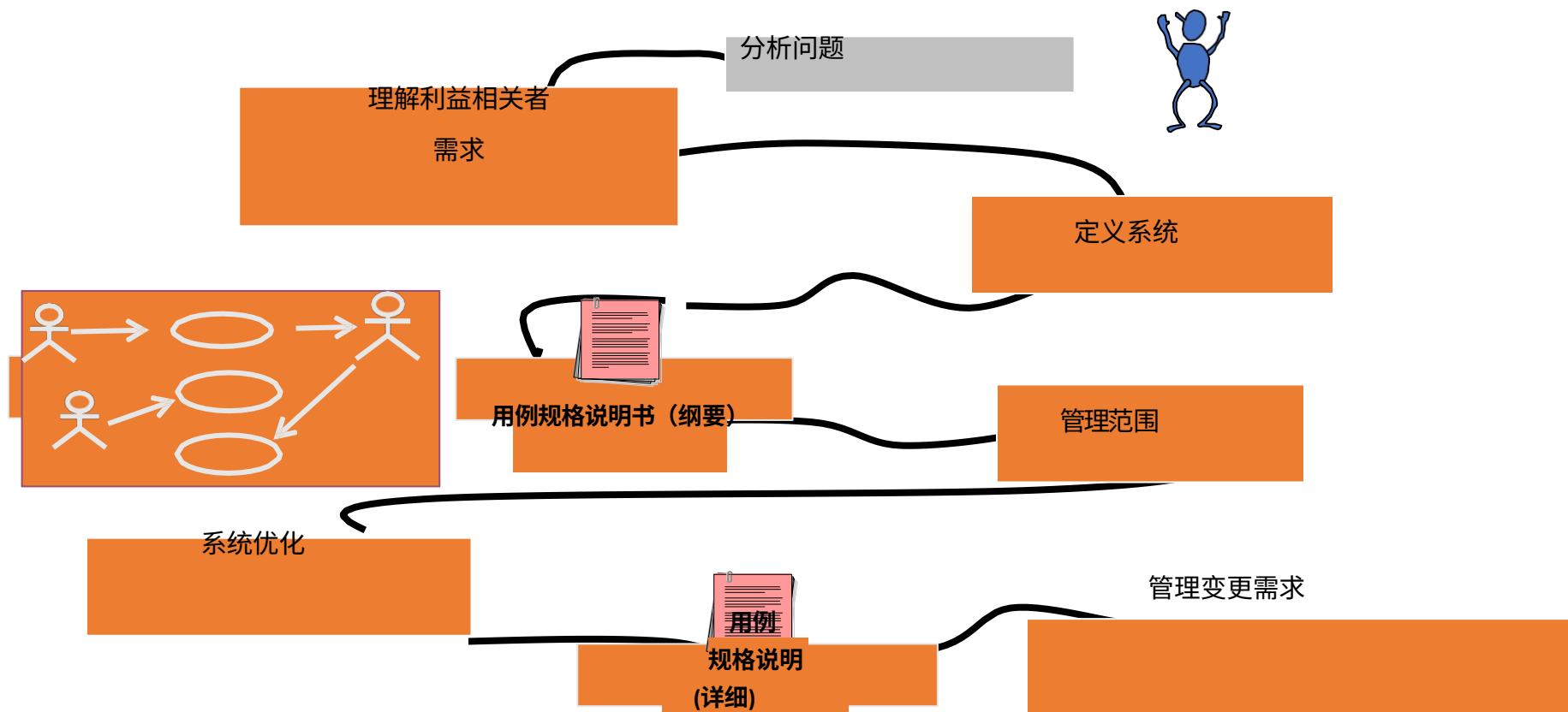
- ◇ 概述所有用例。

- ◇ 对用例流程进行优先级排序。

- ◇ 按优先级顺序详细描述流程。



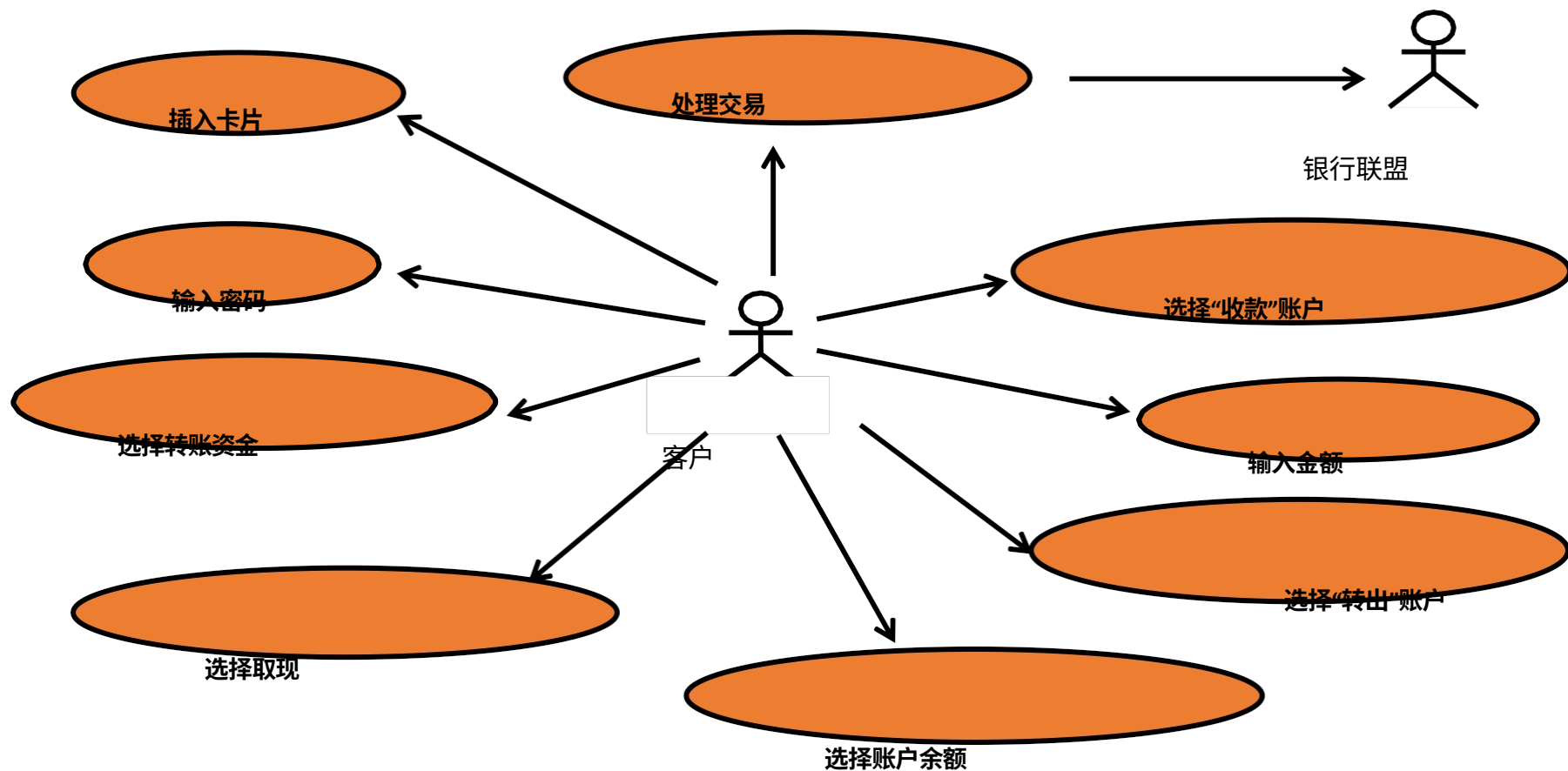
用例在需求管理流程中的定位？



功能分解

- 即将问题分解为小型、独立的部分。
 - 这些部分协同工作以实现系统的功能。
 - 这些部分在孤立状态下通常没有意义。
- 用例：
 - 并非功能分解。
 - 保持功能整体性以描述系统的完整使用场景。
 - 提供上下文背景。

功能分解示例



避免功能分解

症状

- 用例数量极少
- 用例过多
- 无价值结果的用例
- 低级操作命名
 - “操作”+“对象”
 - “函数” + “数据”
 - 示例：“插入卡片”
- 整体模型理解困难

纠正措施

- 寻找更广阔的背景

“为何要构建这个系统？”

- 设身处地为用户着想

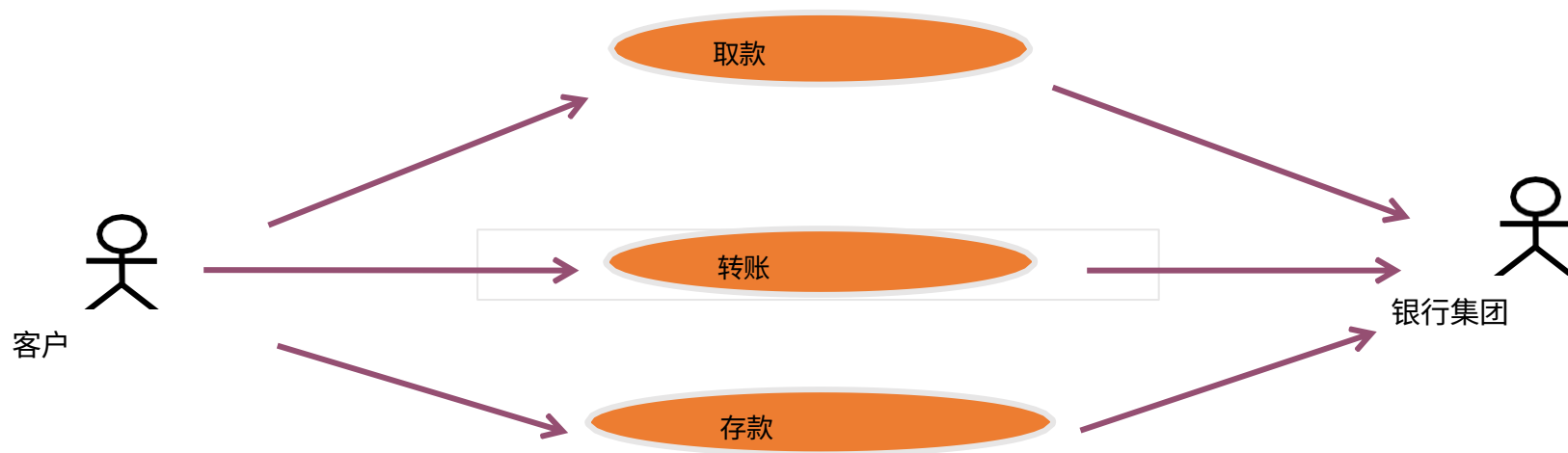
“用户希望实现什么目标？”

“该用例满足谁的目标？”

“此用例增添了什么价值？”

“该用例背后的故事是什么？”

功能分解：修正示例



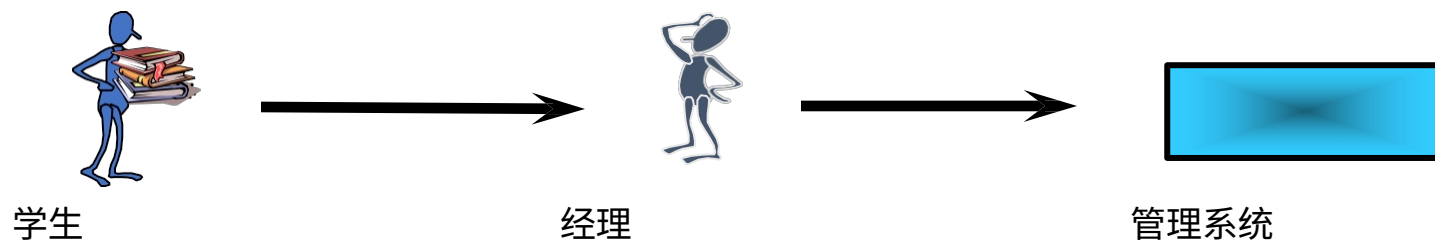
创建用例模型的步骤

- ◇ 确定参与者和用例。
 - ◇ 识别并简要描述参与者。
 - ◇ 识别并简要描述用例。
- ◇ 编写用例。
 - ◇ 概述所有用例。
 - ◇ 对用例流程进行优先级排序。
 - ◇ 按优先级顺序详细描述流程。

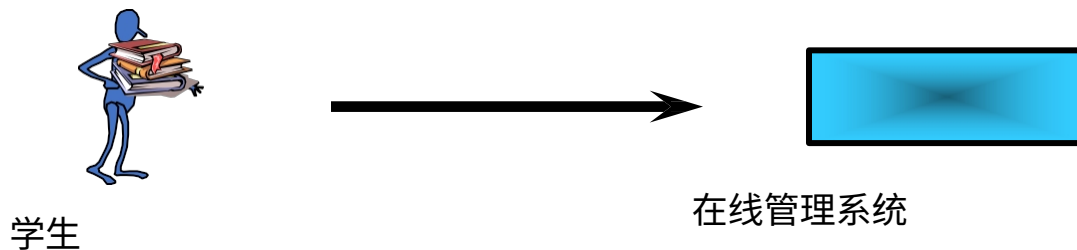


确定参与者

谁在按键（与系统交互）？



学生永远不会接触系统；由管理员操作系统。
或者，您是在构建互联网应用程序吗？



识别参与者

- 谁/什么使用该系统？
- 谁/什么从该系统获取信息？
- 谁/什么向系统提供信息？
- 系统在公司内部使用于哪些部门？
- 谁/什么支持和维护该系统？
- 哪些其他系统使用该系统？

角色描述

文本

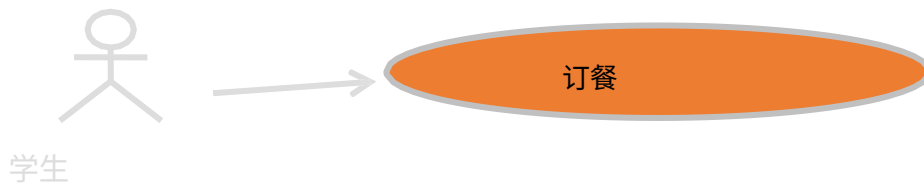
名称

简要描述

与用例的关系

学生

在餐厅用餐的人。

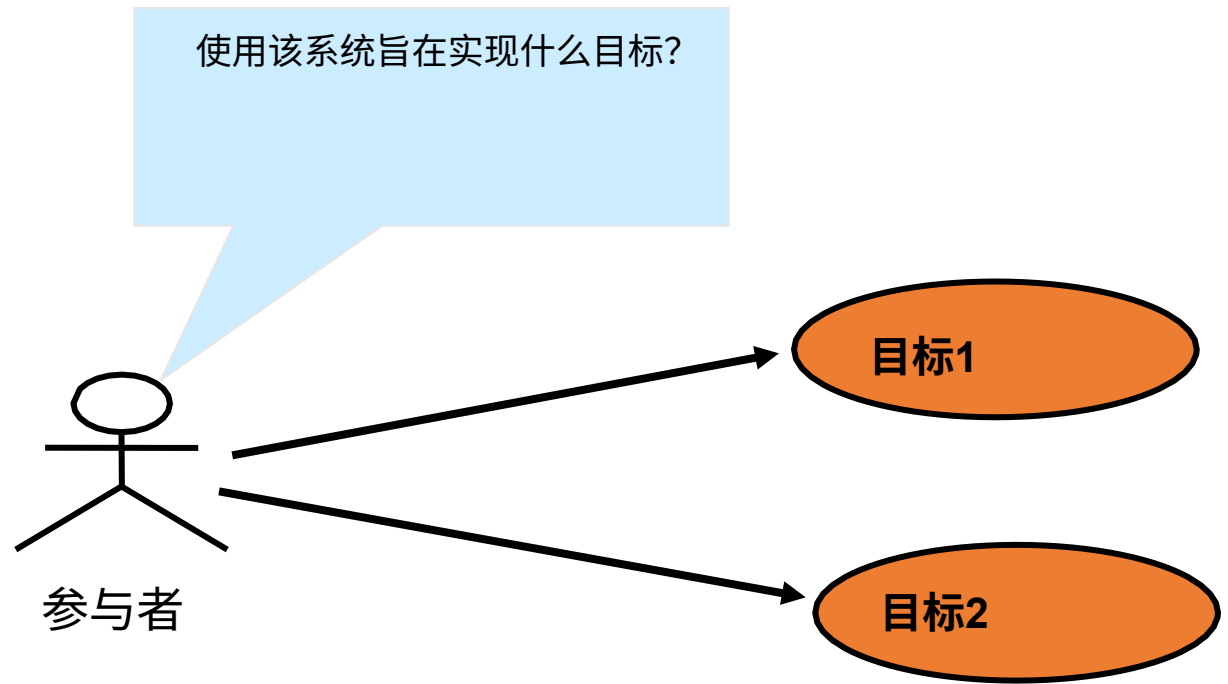


用例模型调查

参与者检查点

- 是否已识别所有参与者？是否已考虑并建模系统环境中的所有角色？
- 每个参与者是否至少涉及一个用例？
- 能否列举至少两名能够担任特定参与者的人员？
- 是否存在多个参与者在系统中扮演相似角色？若存在，请将其合并为单一参与者。





寻找用例

寻找用例的方法

- 与用户交互。
- 基本策略：把自己当作参与者，与设想中的系统进行交互。考虑：
 - 系统交互的目的是什么？
 - 需要向系统输入什么信息？
 - 希望系统执行什么处理并获得什么结果？
- **确定用例与确定参与者不能完全割裂。**

- Jacobson提出以下问题：
 - 参与者的主要任务是什么？
 - 参与者需要了解系统的哪些信息？需要修改系统的哪些信息？
 - 参与者是否需要将系统外部的变化通知系统？
 - 参与者是否希望系统将异常情况的变化通知自己？
- 寻找用例时需要注意的问题：
 - 不要一开始就试图捕捉所有细节。
 - 全面认识和定义每个用例，**关键是以穷举的方式考虑每个参与者与系统的交互情况。**



识别用例

- 每个参与者的目标是什么？
 - 参与者为何需要使用系统？
 - 参与者是否会在系统中创建、存储、更改、删除或读取数据？
如果是，**原因是什么**？
 - 参与者是否需要向系统报告外部事件或变更？
 - 参与者是否需要获知系统中的特定事件？
- 系统是否为业务提供了所有正确的行为？

用例描述

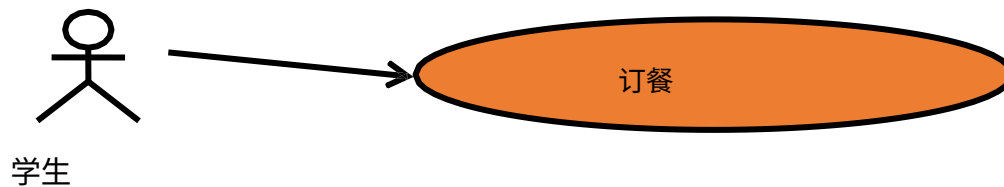
用例的文本描述。名称

订餐

简要说明

学生选择想要食用的餐品

与参与者的关系



用例检查点

- 用例模型呈现系统的行为；通过审查该模型，可以轻松理解系统的功能。
 -
- 所有用例均已识别；这些用例共同涵盖了所有必需的行为。
- 所有功能均映射至至少一个用例。
- 用例模型不含冗余行为；所有用例均可追溯至具体功能需求而得到合理化论证。
- 所有**CRUD**用例均已移除。
 - 创建、检索、更新、删除



用例图的建立步骤

- (1) 确定系统外部的参与者和外部系统，界定系统边界与范围；
- (2) 确定每个参与者期望的系统行为；
- (3) 将这些系统行为命名为用例；
- (4) 运用泛化、包含、扩展等关系处理系统行为的公共或变更部分；
- (5) 为每个用例编写脚本；
- (6) 绘制用例图；
- (7) 区分主事件流与异常情况事件流，可将异常情况事件流作为独立用例处理；
- (8) 细化用例图，解决用例间的重复与冲突问题。

简洁用例示例- Briefly Described Use Cases

● 简洁用例：

简洁的摘要段落，主要描述成功场景

● 处理销售：

客户带着要购买的货物到达收款处，出纳员使用POS系统记录每个购买的货物。系统提供总价和详细条目。客户输入付款信息供系统验证并记录。系统更新库存，客户得到收据并带着货物离开。

临时用例示例- Outlined Use Cases Example

- 临时用例：
 - 非正式、随意的格式
 - 非正式段落，涵盖各种场景

退货处理

主要成功场景：

客户带着要退的货物到达收款处，出纳员使用POS系统记录每件退货商品， ...

候选场景：

若信用验证失败，通知客户并要求使用其他付款方式

若系统检测到与外部计税系统通信失败， ...

详细用例示例- 完整描述的用例示例

用例描述

名称：下单

前提条件：有效用户已登录系统。描述：

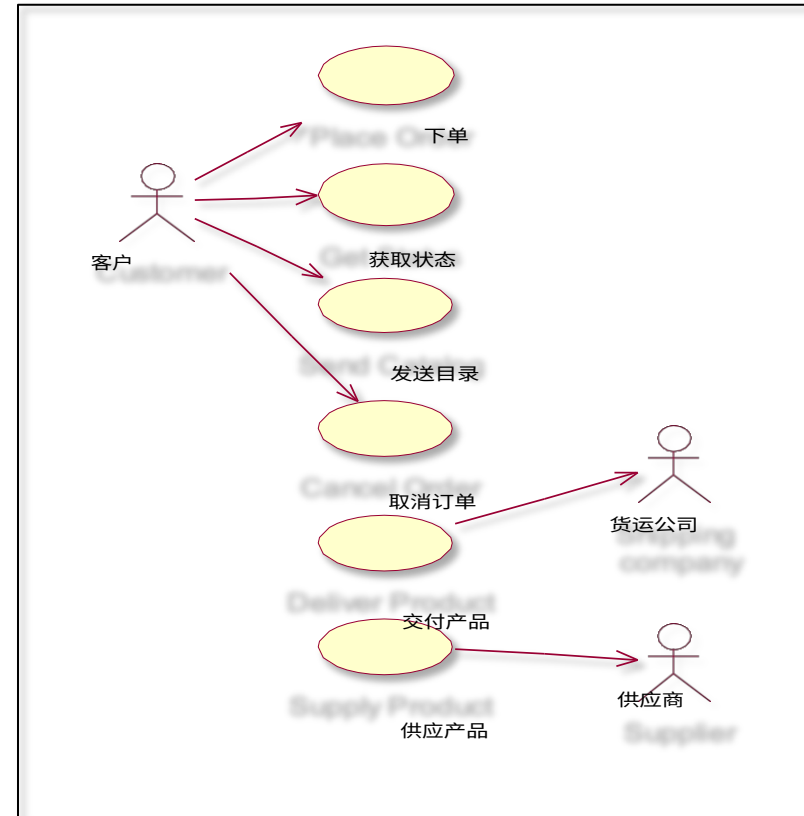
1. 用例始于客户选择“下单”。
2. 客户输入姓名和地址。
3. 若客户仅输入邮政编码，系统将自动补全城市及州信息。
4. 客户需输入所需商品的货号。
5. 系统将为每件商品显示产品描述及价格。
6. 系统将实时累加所订商品的总金额。
7. 客户需输入信用卡支付信息。
8. 客户点击提交按钮。
9. 系统将验证信息，将订单保存为待处理状态，并将付款信息转发至会计系统。
10. 当付款确认后，订单状态将标记为“已确认”，系统向客户返回订单编号，用例流程结束。

异常处理：

在步骤9中，若任何信息有误，系统将提示客户修正信息。

后置条件：订单已在系统中保存并标记为确认状态。

用例图



主成功流程

1. 顾客插卡
2. ATM读卡
3. ATM提示选择语言
4. 顾客选择英语
5. ATM提示输入密码
6. 顾客输入密码
7. ATM显示待选服务
8. 顾客选择取款
9. ATM提示输入取款金额，必须为50的倍数
10. 客户输入数值
11. ATM通知银行中央系统客户的取款金额
12. 银行中央系统接受请求，并通知ATM新的账户余额
13. ATM输出现金
14. ATM询问顾客是否需要收据
15. 客户回答需要收据
16. ATM输出收据并打印
17. ATM记录该项交易

例外情况：

- 读卡器故障或卡片刮伤 读卡异常
- 非本行发行的卡
- 密码错误
- 客户未及时输入密码
- ATM 故障
- 主机故障或网络故障
- 账户余额不足
- 客户未及时输入金额输入金额超时
- 金额不是50的倍数
- 金额过大
- 交易期间网络或主机故障
- ATM现金不足
- 取款时现金卡纸
- 收据纸用完或卡纸
- 顾客未从取款机取走现金

非本机接受卡种

错误密码

输入密码超时

机器停机

主机坏或网络断

账户余额不足

金额不符合要求

交易期间网络或主机故障

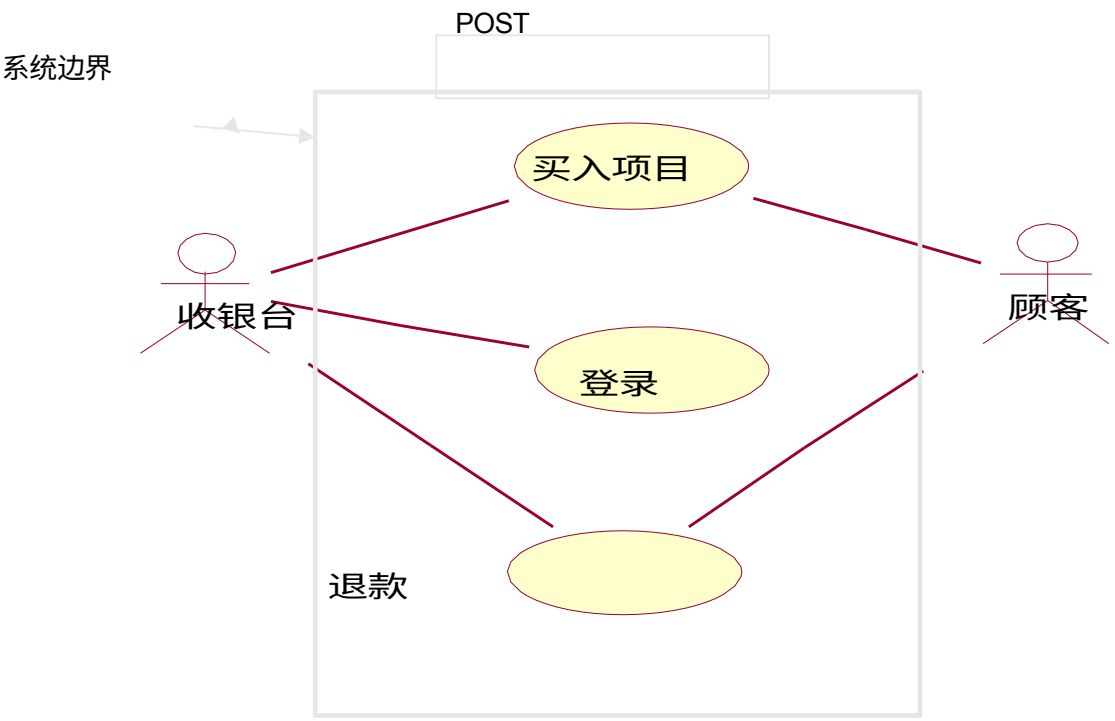
ATM现金量不足

出闭口夹纸

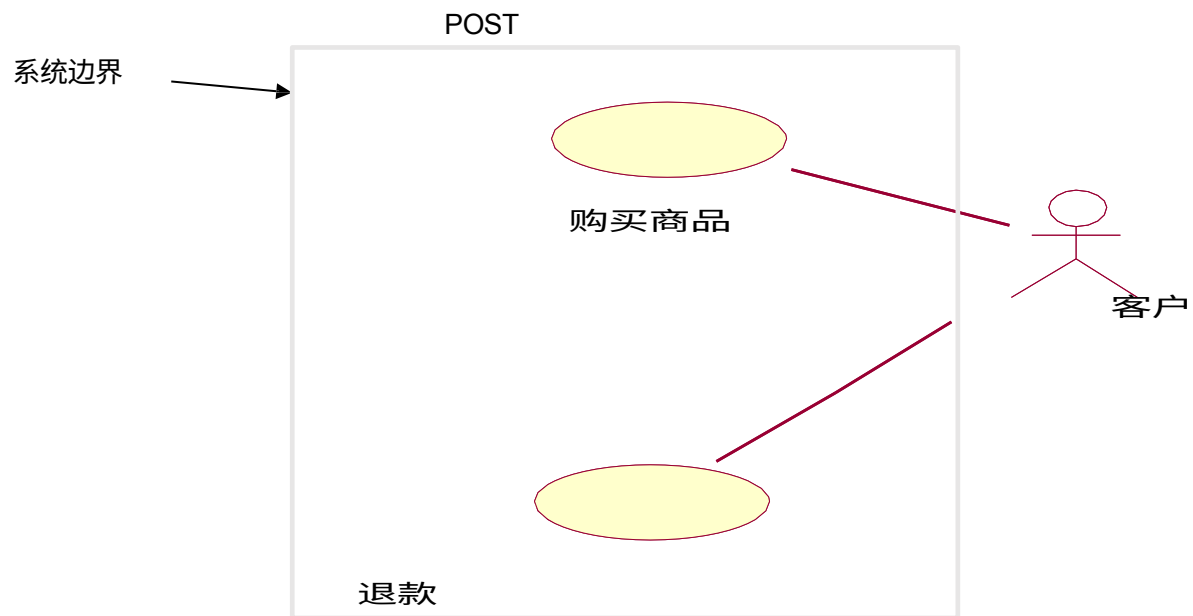
设定系统边界

- 系统边界会对用例以及Actor的定义有所影响
- 思考以下系统的用例图：
 - 记录销售及付款情况的计算机系统
 - 用于零售店
 - 包括硬件设备，如计算机、条码扫描装置
 - 运行在系统上的软件
 - 系统目标包括：
 - 自动收款
 - 快速准确的销售情况统计及分析
 - 自动库存管理

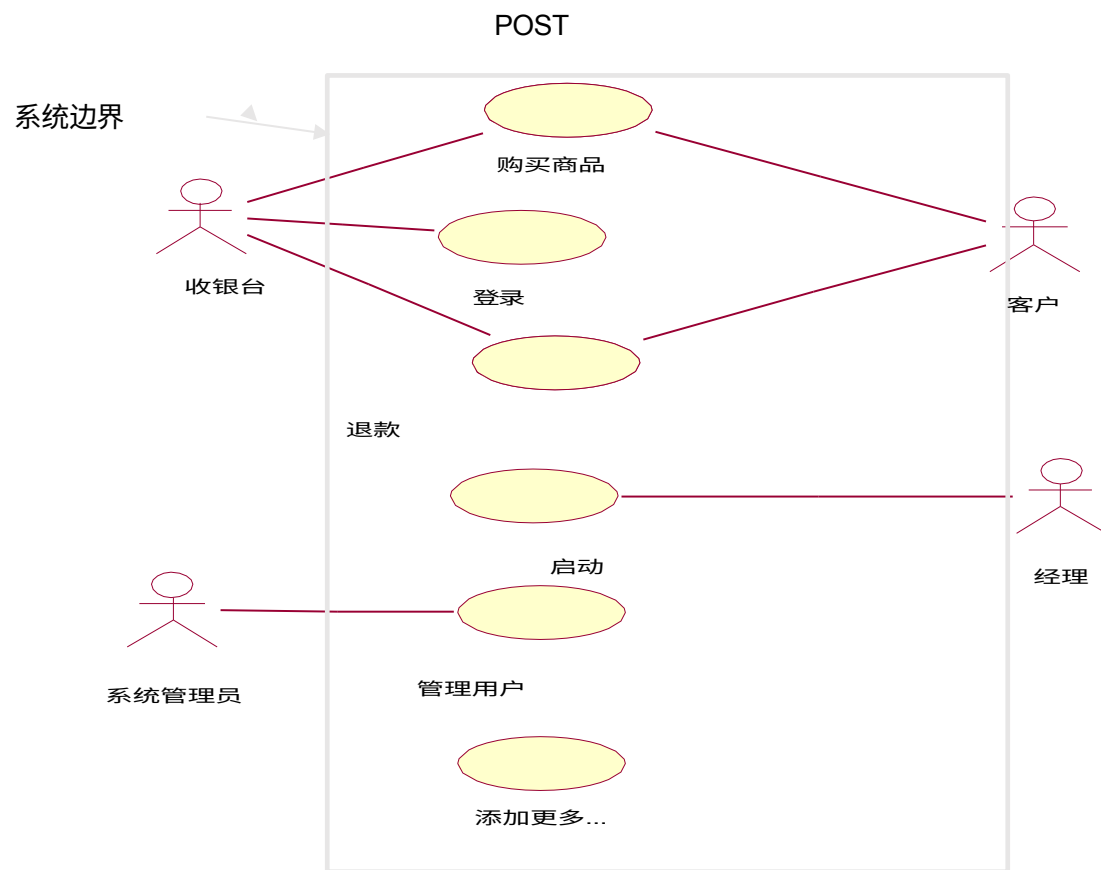
系统边界（系统边界定义之一）



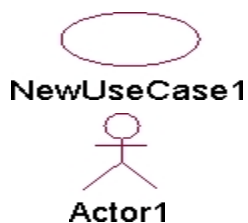
系统边界定义之二



系统边界定义之三



用例图中的一些主要图标



关联

关联

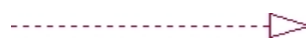
泛化依赖

说明：UML中不使用颜色作为图形语义的区分标记。



注释

注释连接



实现



用例实现

<<extend>>

扩展用例包含用例

<<include>>

用例之间的关系

- 用例除了与参与者**存在关联关系**外，用例之间也存在特定关系，包括：**泛化关系**、**包含关系**、**扩展**关系等。
- 也可利用UML的扩展机制自定义用例间的关系。
- **关系**、**关联**、**泛化**、依赖的区别。
 - 关联、泛化、依赖都属于关系。
 - include、extend 属于依赖关系。

泛化关系

- **泛化(generalization)**代表一般与特殊的关系。用例间的泛化关系与类之间的泛化关系（继承关系）类似。
- **用例泛化是用例**（子类）与用例（父类）之间的分类关系，描述子类与具有相同父类的其他用例共享的特征。

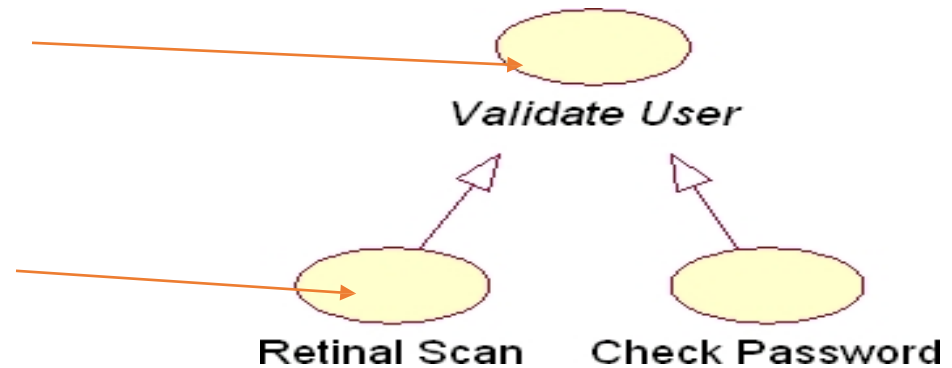
泛化示例：

父用例"检查密码"的行为：

1. 从主数据库获取密码
2. 向用户请求密码
3. 用户输入密码
4. 根据用户输入验证密码

父用例

子用例



子用例行为：视网膜扫描

1. 从主数据库获取视网膜特征
2. 扫描用户视网膜并获取特征
3. 将主数据库签名与扫描签名进行比对

关于泛化的更多说明

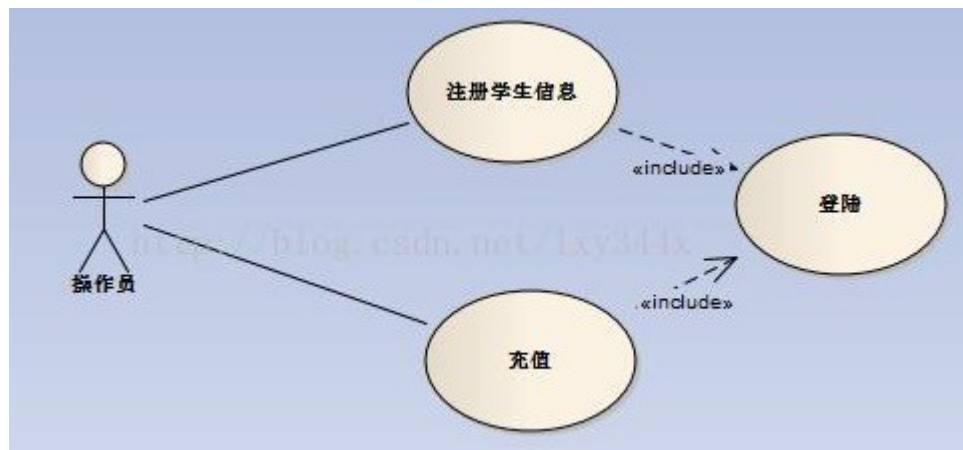
- 子用例继承父用例的行为与含义
父用例的行为和含义；
- 子用例可扩展或覆盖父用例的行为与含义；
- 子用例可在父用例出现的所有位置进行替换。

包含关系

- 包含关系是指一个基本用例的行为包含了另一个用例的行为。
- 从基础用例到包含用例的关系，指定如何将包含用例定义的行为插入到基础用例定义的行为中。

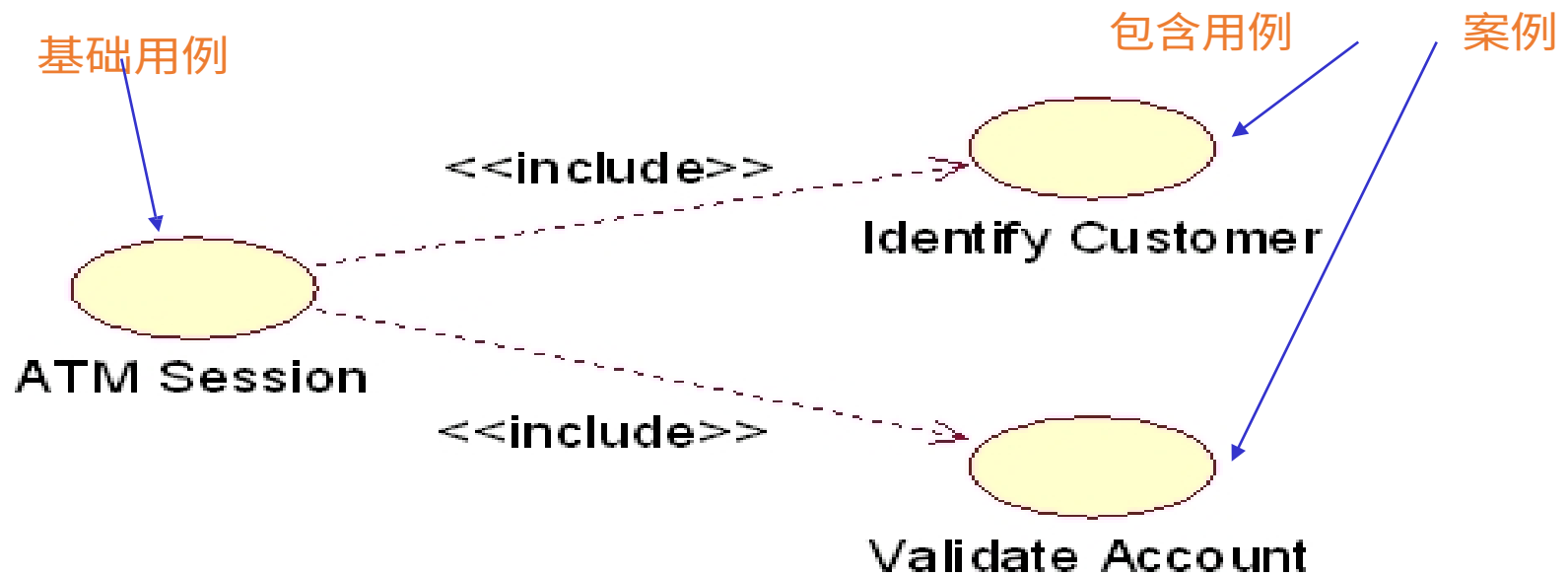
何时使用包含关系？

- 当多个用例存在共享行为时
- 将其拆分为独立用例，并为其他用例
"包含"该行为（为共享行为单独创建用例）
- 示例包括
 - 登录



<https://blog.csdn.net/lxy344x/article/details/40346525>

- 包含关系的例子：



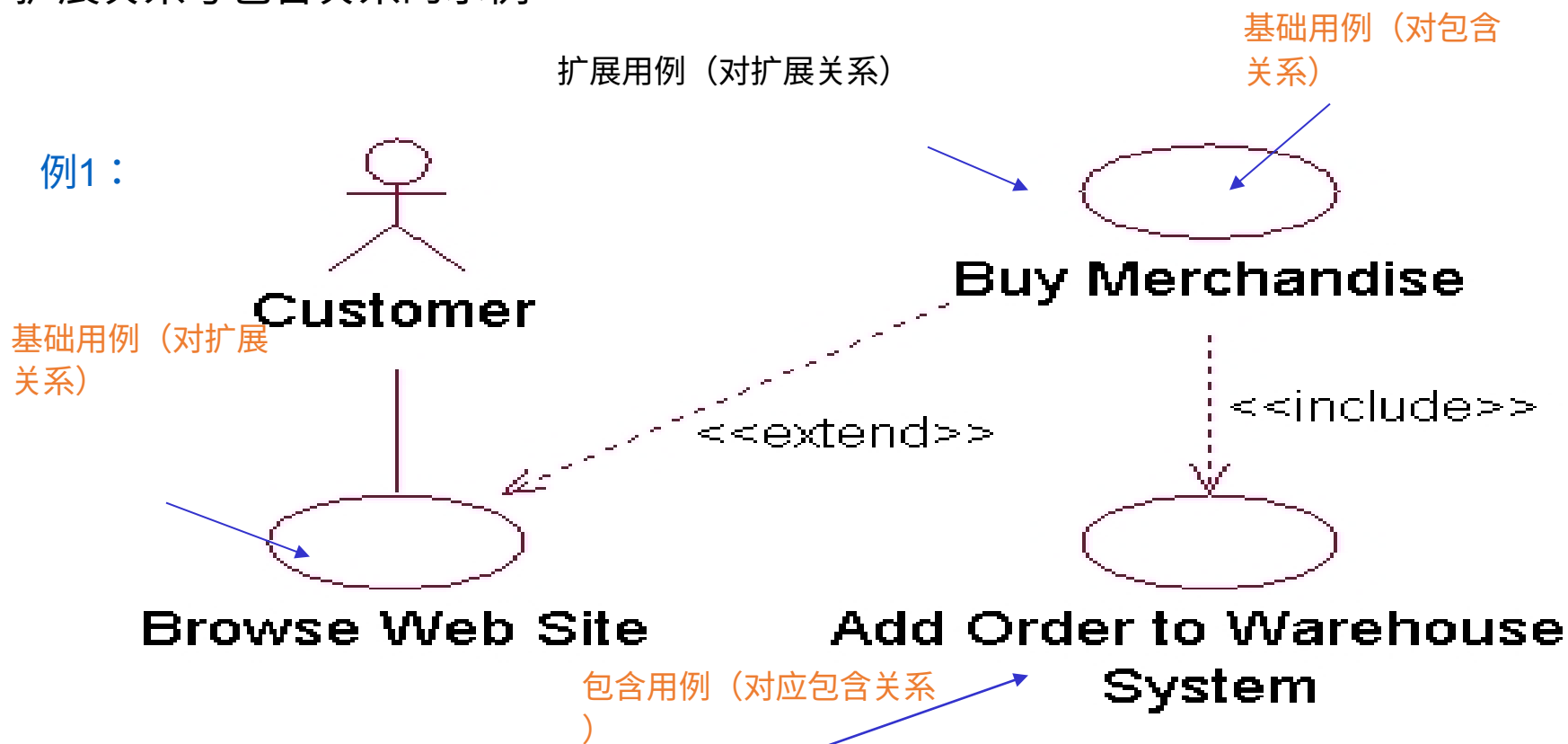
扩展关系

- **扩展关系**的基本含义与泛化关系类似，如果一个用例明显地混合了两种或两种以上的不同场景，即根据情况可能发生多种分支，则可以将这个用例分为一个基本用例和一个或多个扩展用例。
- 从**扩展用例**到**基础用例**的关系，指定如何将**扩展用例**定义的行为插入到**基础用例**定义的行为中。

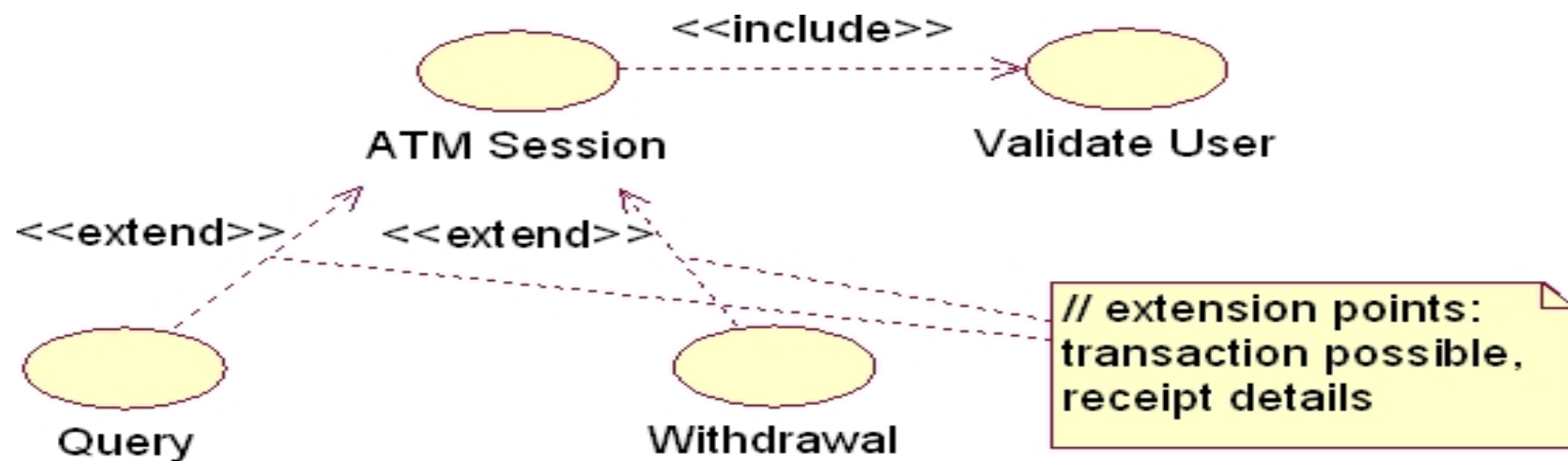
何时使用扩展关系？

- 当一个用例与另一个用例相似，但包含少量额外活动时
- 将常规行为定义为一个用例，特殊行为定义为扩展用例

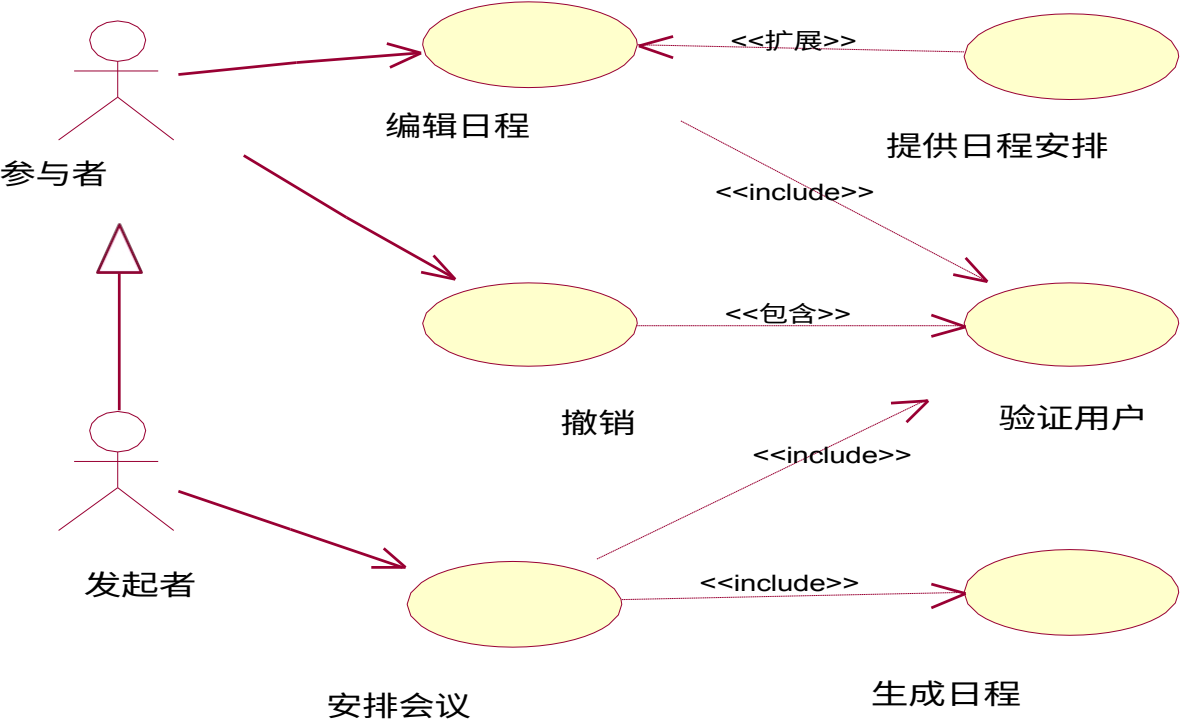
扩展关系与包含关系的示例：



例2：



例3：

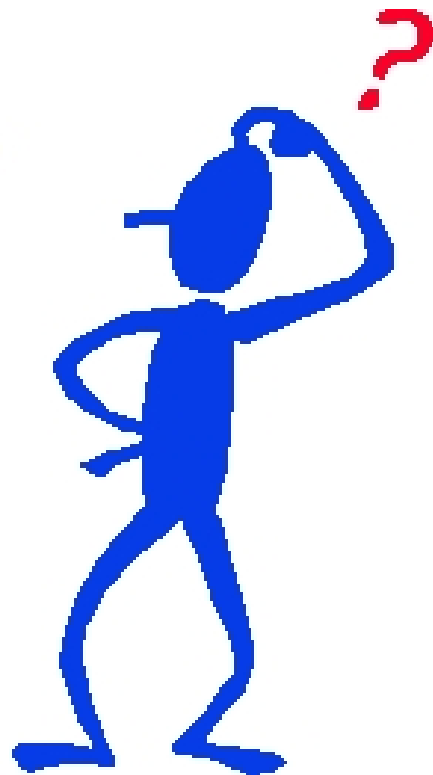


常见问题分析

1. 用例粒度问题，即对于一个系统的用例图，所包含的用例数量问题。

- 这是很多人争论的焦点。例如，Jacobson认为，对于一个十人年的项目，他需要二十个用例。而在一个相同规模的项目中，Martin Fowler却使用了一百多个用例。

* 《UML精要》一书的作者



2. 许多应用程序需要对系统访问进行控制，登录问题应如何处理较为妥当？

- 存在四种处理登录用例的方式：

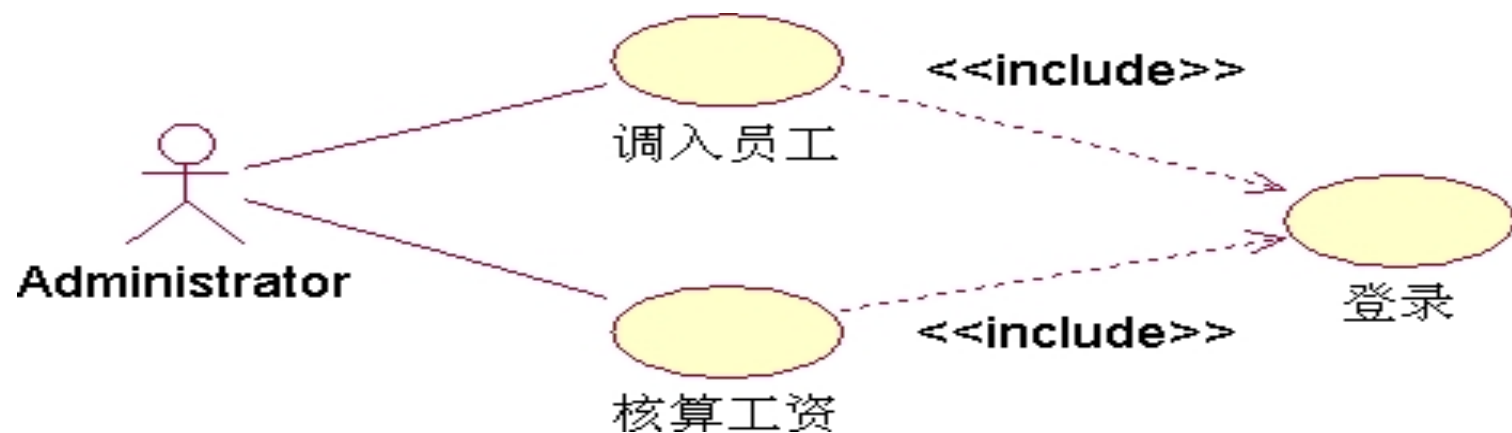
(1) 其他用例包含登录；

(2) 其他用例扩展登录；

(3) 登录独立于其他用例；

(4) 登录包含其他用例。

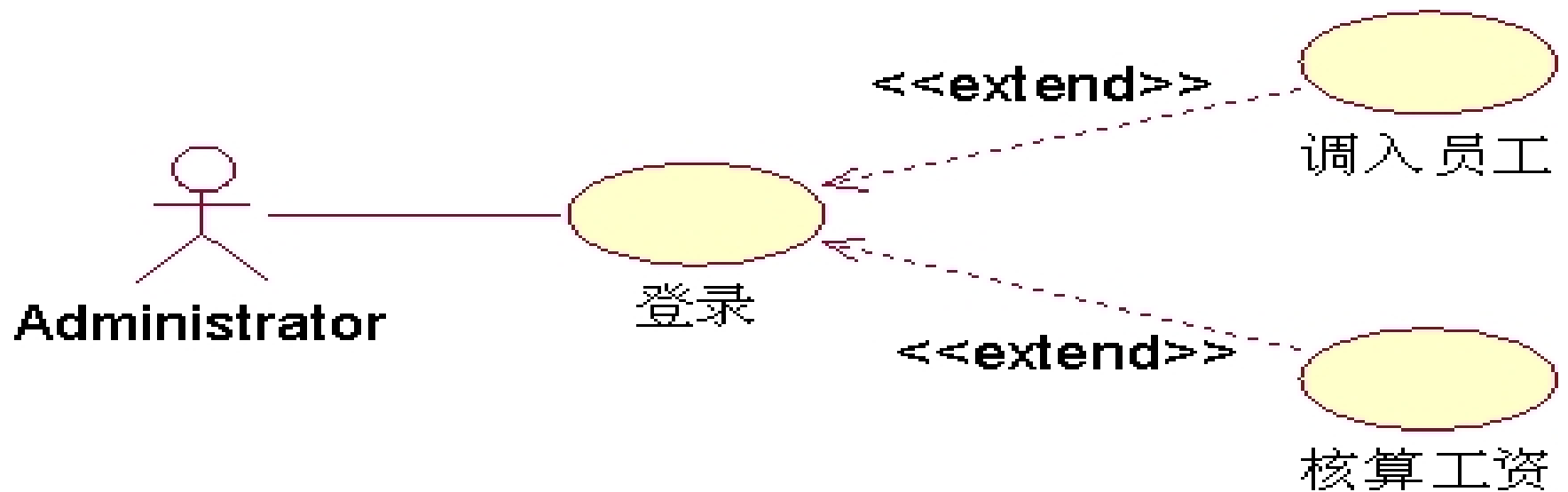
(1) 其他用例包含登录



用例说明：

这种处理方式的特点：

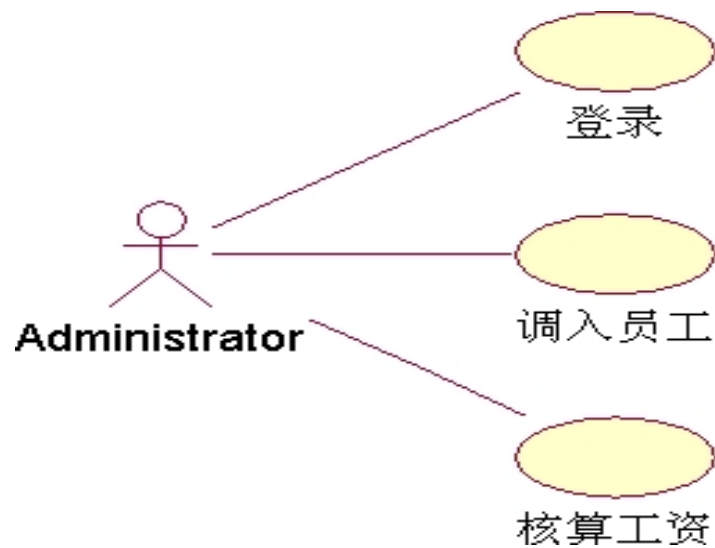
(2) 其他用例扩展登录



用例说明：

这种处理方式的特点：

(3) 登录独立于其他用例。



用例说明：

这种处理方式的特点：

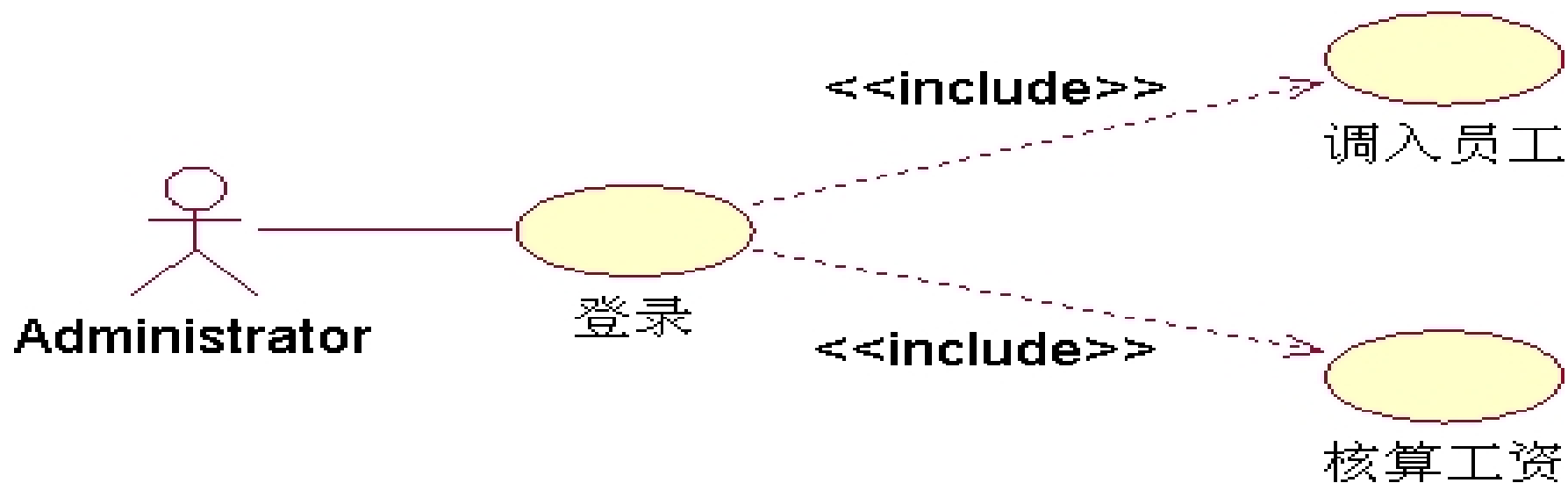
- 登录用例说明:

1. 当超级用户启动应用时用例开始
2. 系统提示超级用户输入用户名和密码
3. 超级用户输入用户名和密码
4. 系统验证其是否为有效超级用户
5. 用例结束

- 调入员工用例说明:

前置条件：一个有效的超级用户已登录系统

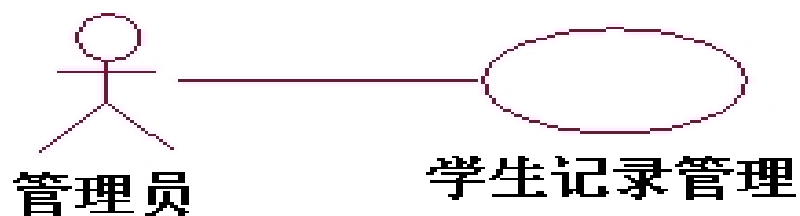
(4) 登录包含其他用例



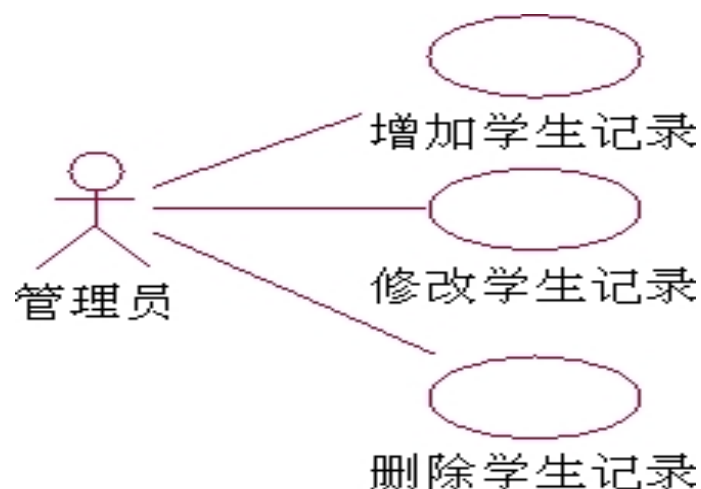
用例说明:

存在的问题:

3. 假设有这样需求：学生档案管理中，用户经常需要做三件事：增加一条学生记录、修改一条学生记录、删除一条学生记录。若需绘制用例图，以下两种方法哪种更合适？



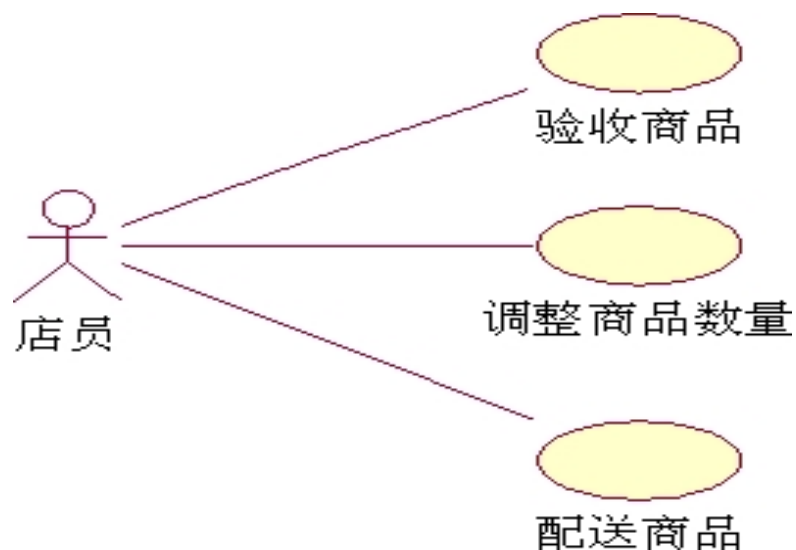
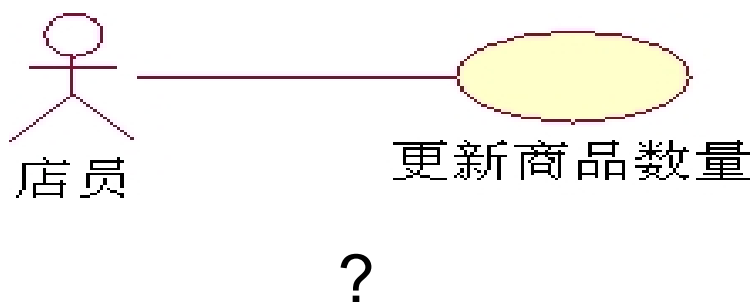
方法1：再分成3个脚本，分别画3个交互图。
脚本1 增加学生记录；脚本2 修改学生记录；
脚本3 删除学生记录。



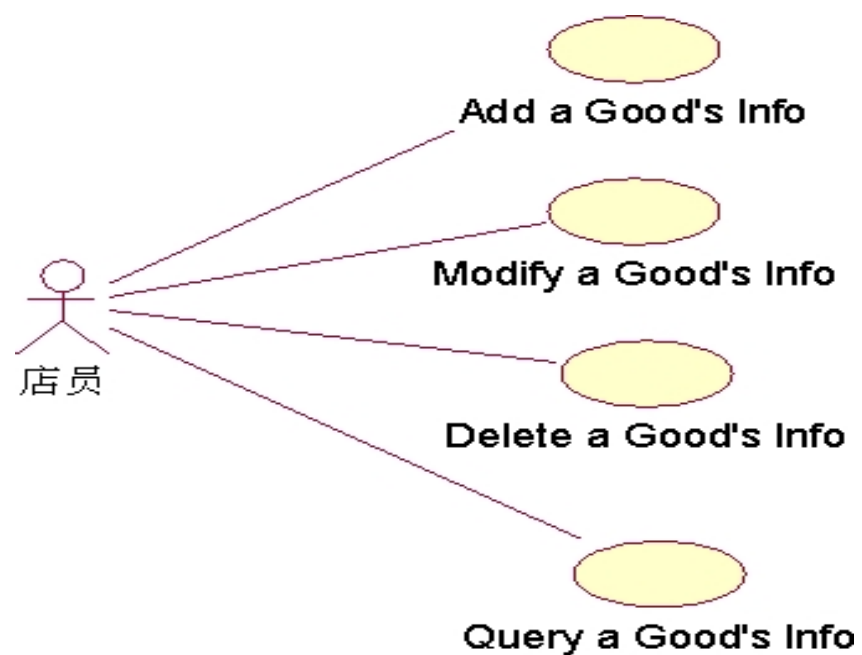
- 创建、检索、更新、删除类型用例的处理：

- 采用CRUD四个用例还是一个用例？
- 从用户需求角度考虑而非数据处理角度考虑。

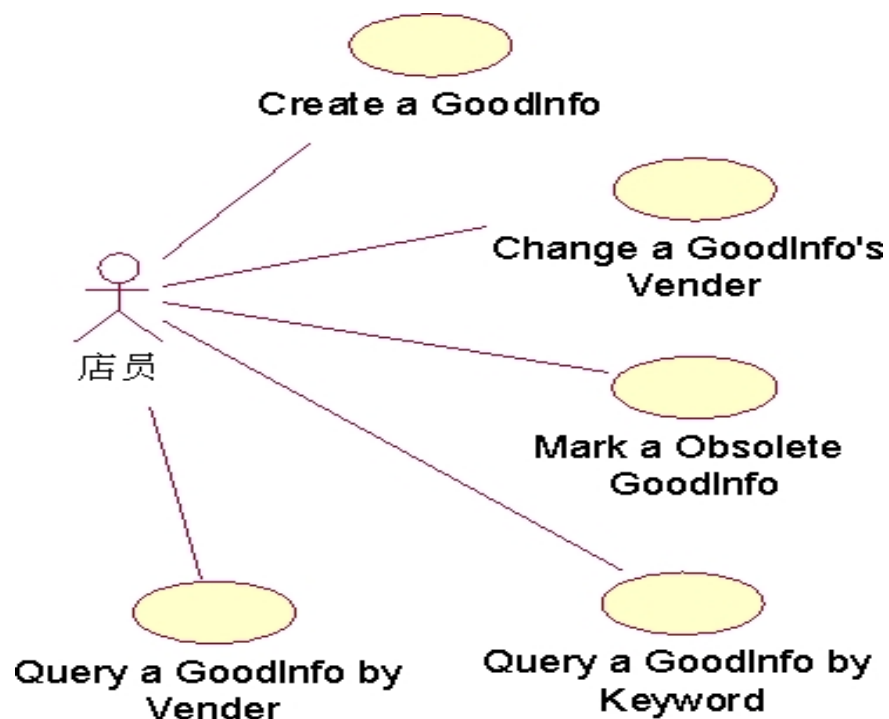
- 例2：



• 例3:



?



课堂练习 问题描述

- 我们在为一家邮购公司开发订单处理软件，从供应商处采购产品，再进行销售。
- 该公司每年发布两次产品目录，并将其邮寄给客户及其他感兴趣的人士。
- 客户通过提交商品清单并向邮购公司付款的方式购买商品。邮购公司开具账单并将商品运送到客户地址。
- 订单处理软件记录从收到订单到商品被运送给客户的整个过程。
- 邮购公司将提供快捷的服务，以最快、最有效的方式运送客户订购的产品。
- 客户可以退货或要求补货，但有时需要支付费用。

确定执行者——Actor

- 谁使用这个系统？
- 谁安装这个系统？
- 谁启动这个系统？
- 谁维护系统？
- 谁关闭系统？
- 哪些其他系统使用这个系统？
- 谁从这个系统获取信息？
- 谁为这个系统提供信息？
- 是否有事情自动在预定时间发生？

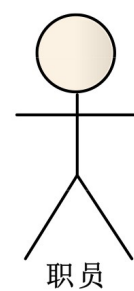
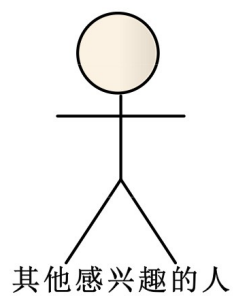
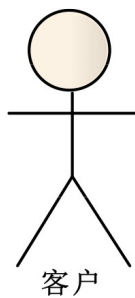
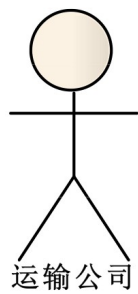
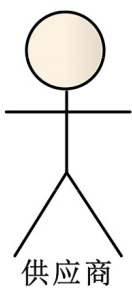
利益相关者(Stakeholder)?

主要参与者、次要参与者? 系统? 无关?

- 我们 (开发人员)
- 邮购公司
- 供应商
- 产品
- 产品目录
- 客户
- 特快专递
- 其他感兴趣的人
- 商品列表
- 运输公司
- 订单处理软件
- 邮购公司职员
- 玛丽·史密斯 (邮购公司客户)
- 玛丽·史密斯 (邮购公司职员)
- 记账系统
- 库存系统

参与者

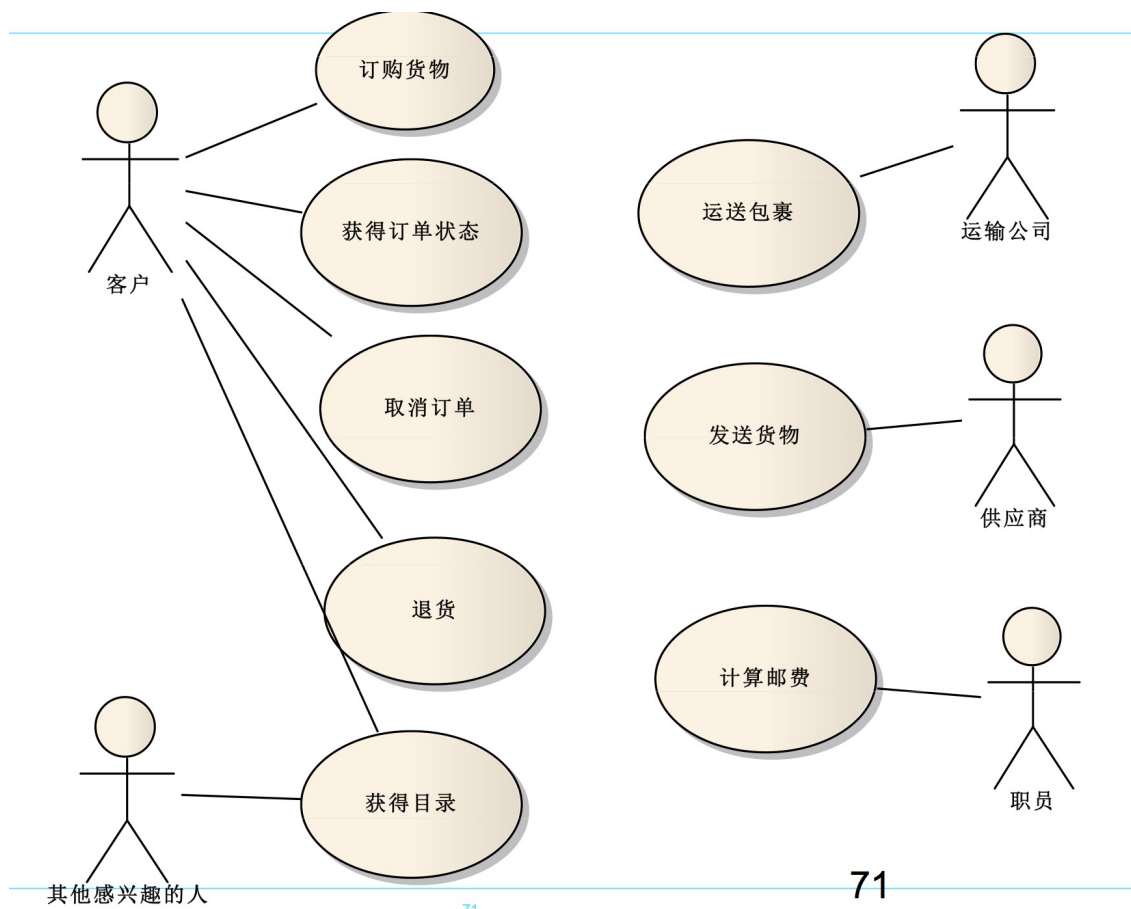
uc Actors



确定用例

- 执行者希望系统提供什么样的功能？
- 系统存储信息吗？
- 执行者将创建、读取、更新或删除哪些信息？
- 系统是否需要将自身内部状态的变化通知给执行者？系统必须知道哪些外部事件？执行者将如何通知系统这些事件？
- 其他需要考虑的用例包括启动、关闭、诊断、安装、维护、培训和改变商业流程。

用例



UC01：订购货物用例

前置条件：合法用户已登录系统事件流：

1. 当客户选择订购货物时，用例开始；
2. 客户输入想要购买的商品代码；
3. 系统逐项列出产品描述和价格；
4. 系统保存已订购的产品清单；
5. 客户输入信用卡支付信息；
6. 客户选择提交；
7. 系统验证输入信息；
 - a. 若提交信息正确，保存订单并向记账系统转发支付信息；
 - b. 若客户提交信息不正确，系统提示用户修改；
8. 当支付确认后，订单被标记为已确认，同时返回给客户一个订单ID，用例结束。
 - a. 若支付未获确认，系统提示客户修正支付信息或取消；
 - b. 若客户选择修改信息，则返回第5步；
 - c. 若选择取消，用例结束。

后置条件：若订单未被取消，则由系统保存并标记为已确认

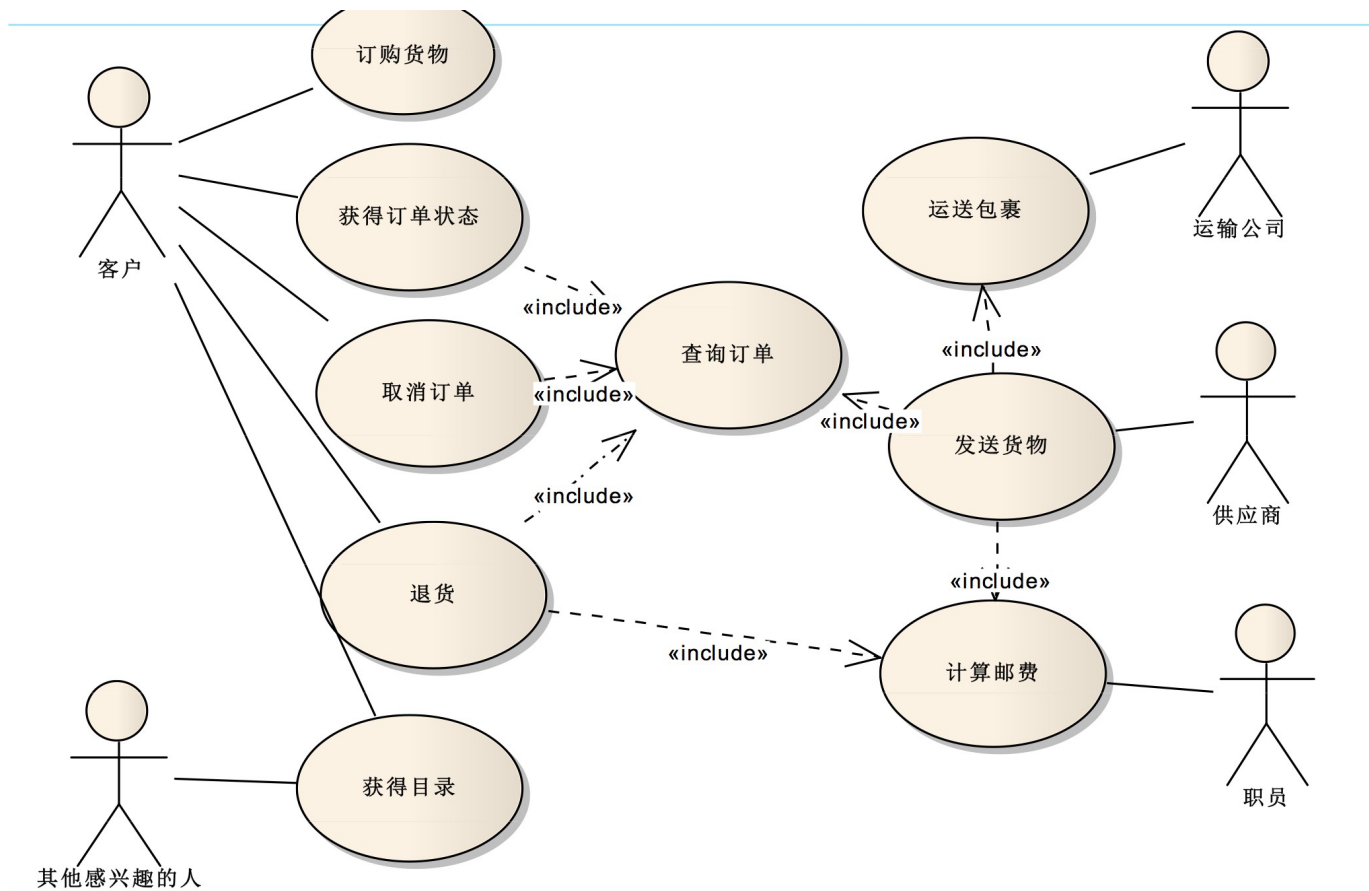
找出可选路径的方法

- 沿基本路径逐条排查，并考虑：
 - 在此节点是否可执行其他活动？
 - 在此节点是否存在潜在错误？
 - 是否有随时可能发生的操作？
- 或者，通过以下大类来发现可选路径。例如：
 - 参与者退出应用程序；
 - 参与者取消指定操作；
 - 参与者请求帮助；
 - 参与者提供了“无效数据”；
 - 参与者提供了不完整数据；
 - 参与者选择了执行用例的可选方法；
 - 系统崩溃；
 - 系统不可用。

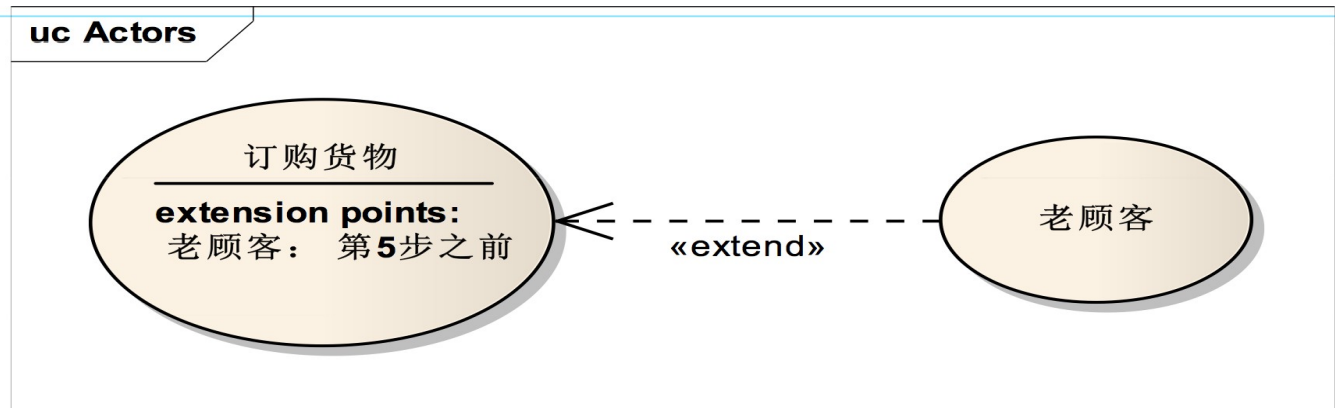
可选路径

- 1. 商品代码不正确；
 - 2. 信用卡支付信息错误；
 - 3. 填写订单过程中，用户选择取消；
 - 4. 用户选择修改订单信息。
-
- 需要使用订单搜索功能的场景包括：
 - 1. 取消订单；
 - 2. 查询订单状态；
 - 3. 退货。

修改后的用例



扩展用例及扩展点



UC012：老顾客折扣用例事件流

：

- 当顾客选择老顾客身份时，用例启动；
- 系统提示客户输入顾客卡号；
- 顾客输入卡号后，系统验证卡号合法时；
- 系统为顾客计算折扣信息。

尝试

- 现急需开发一个“餐厅食品溯源管理系统”
 - 如何对用例进行建模？

相关方(Stakeholder)? 主参与者、次参与者(Actor)? 系统? 无关?

- 学生
- 教师
- 食堂经理
- 供应商
- 窗口阿姨
- 刷卡机
- 校区管理人员
- 服务维护人员
- 点菜系统
- ...

参考

- 清华大学国家级精品课程《软件工程》主讲人刘强副教授 刘璘副教授
- https://www.icourses.cn/sCourse/course_3016.html
- https://www.xuetangx.com/course/THU08091000367/5883555?channel=learn_title



谢谢大家！

THANKS

